SAIR

Spatial AI & Robotics Lab

CSE 473/573

L12: RANSAC & BLENDING

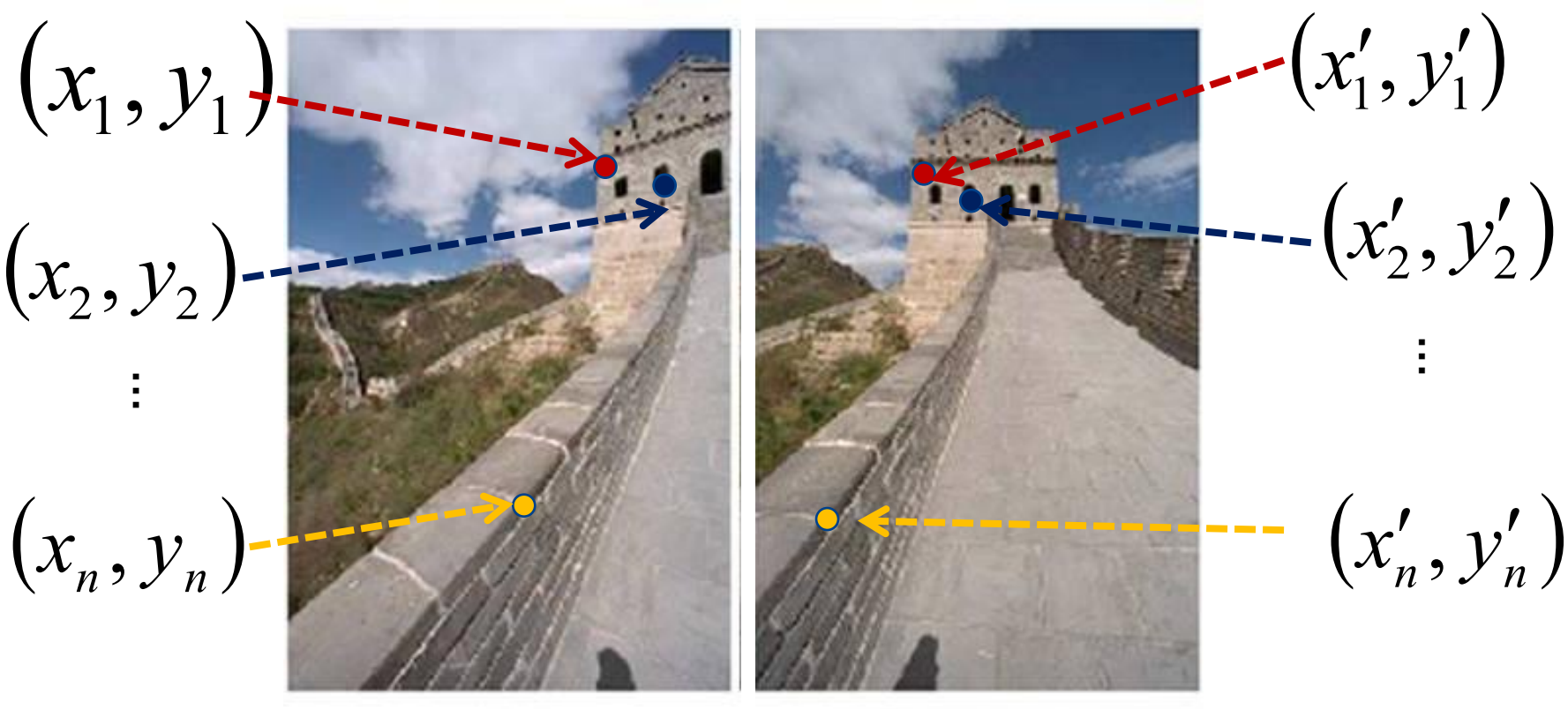
Chen Wang

Spatial AI & Robotics Lab

Department of Computer Science and Engineering

University at Buffalo The State University of New York

Recap: Alignment

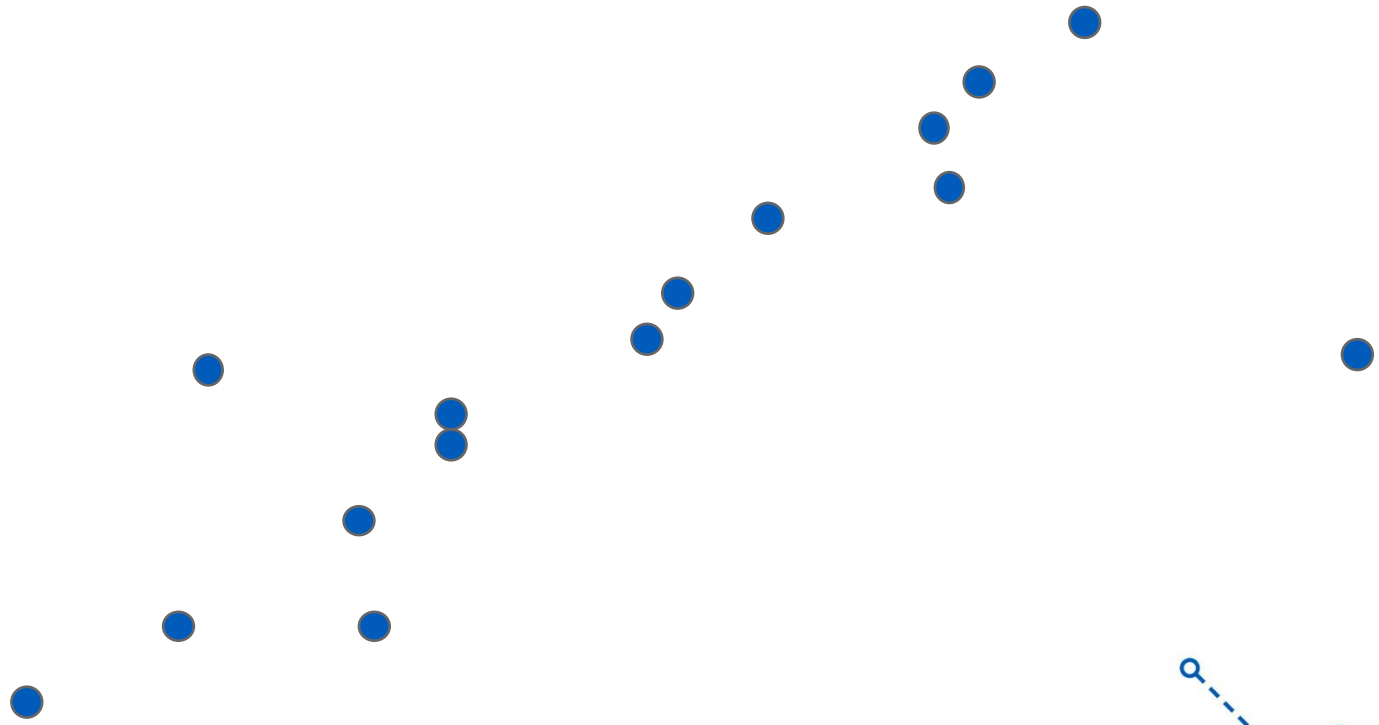


What if there are incorrect matches?

- How can we remove outliers?

Similar Problem: Line Fitting

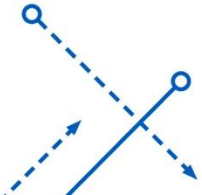
- Not all data may be representative
- There may be “outliers”
- If you use them, your result may not be accurate



Solutions?

- Least Squares Fit?
  - Closed for solution…
  - Sensitive to outliers

- Hypothesize and Test
  - Try out as many lines as we want
  - Keep the best lines
  - But which are the best?

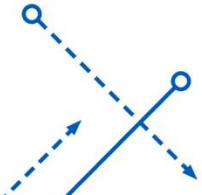


RANSAC

- **RAN**dom **S**ample **C**onsensus
  - An iterative method for estimating a mathematical model from a data set that contains outliers.

- Motivation: we want to avoid the impact of outliers, so let's look for "inliers", and use those only.

- Idea: if an outlier is chosen to compute the current model, then the model won't have much support from rest of the points.



RANSAC

RANSAC loop:

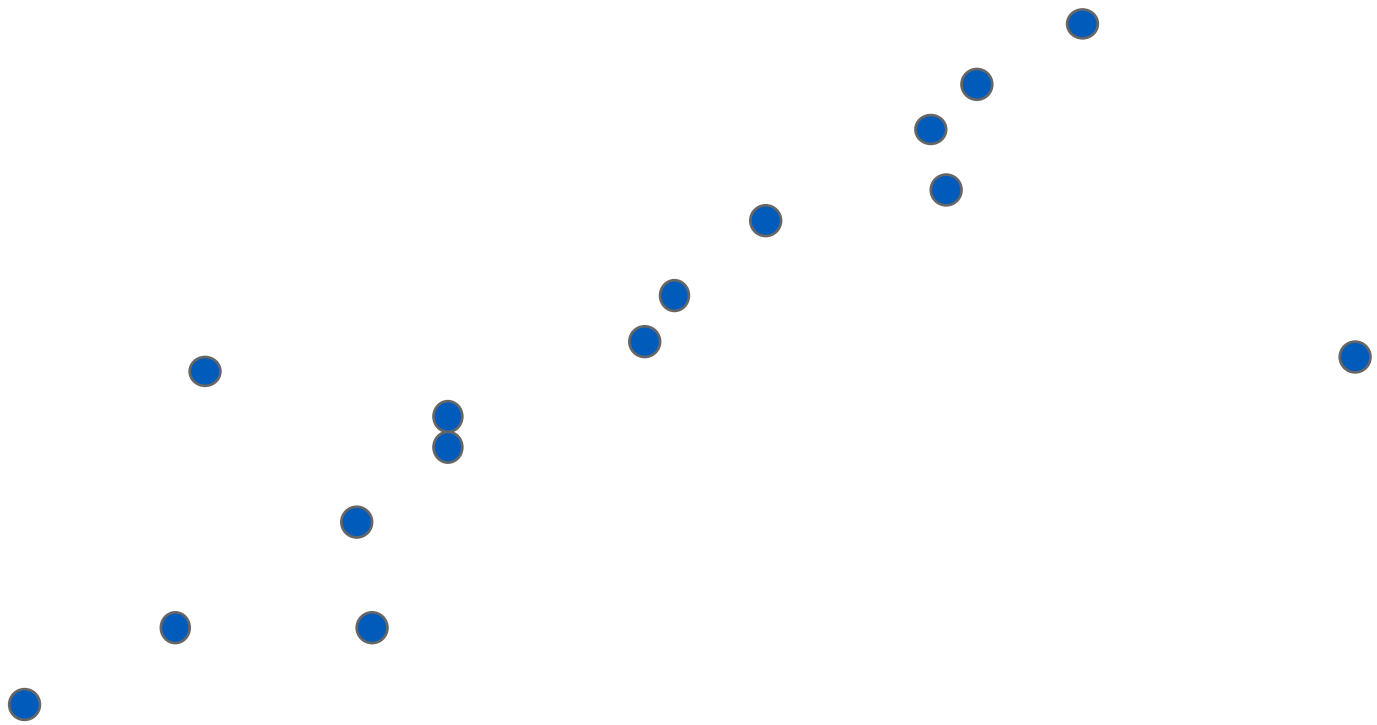
1. Randomly select a **seed group** of points on which to base transformation estimate (e.g., a group of matches)
2. Compute transformation (**model**) from seed group
3. Find **inliers** to this transformation
4. If the number of inliers is sufficiently large, re-compute least-squares estimate on all inliers.

- Keep the model with the **largest number of inliers.**

RANSAC Line Fitting Example

Task:

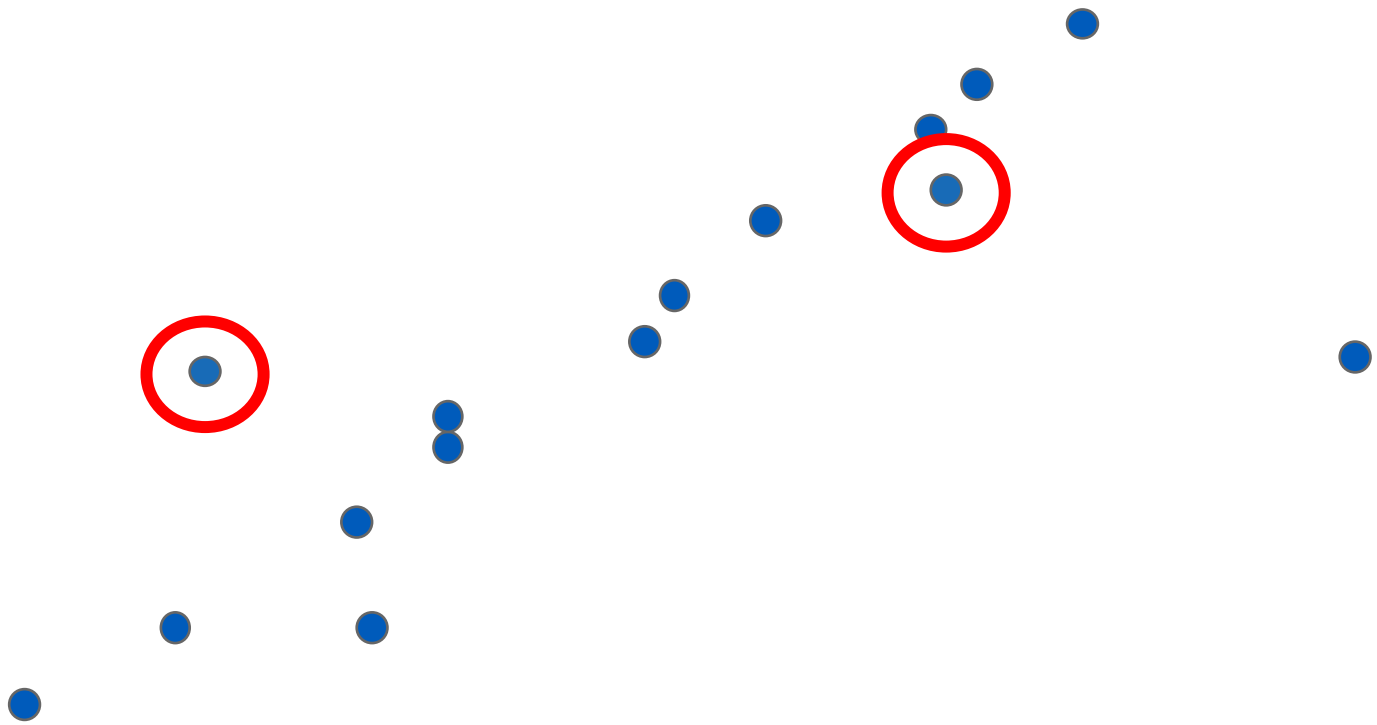
Estimate best line



Slide credit: Jinxiang Chai, CMU

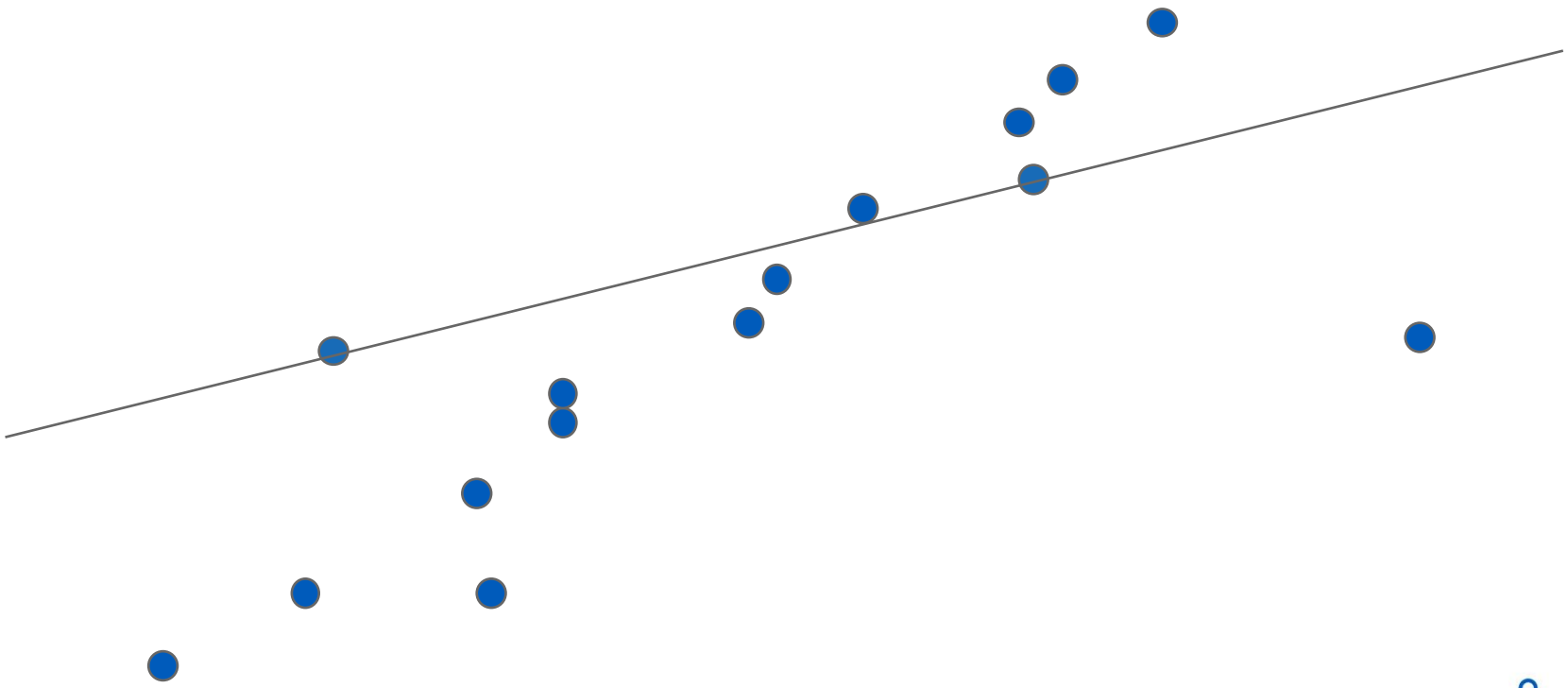
RANSAC Line Fitting Example

Sample two points



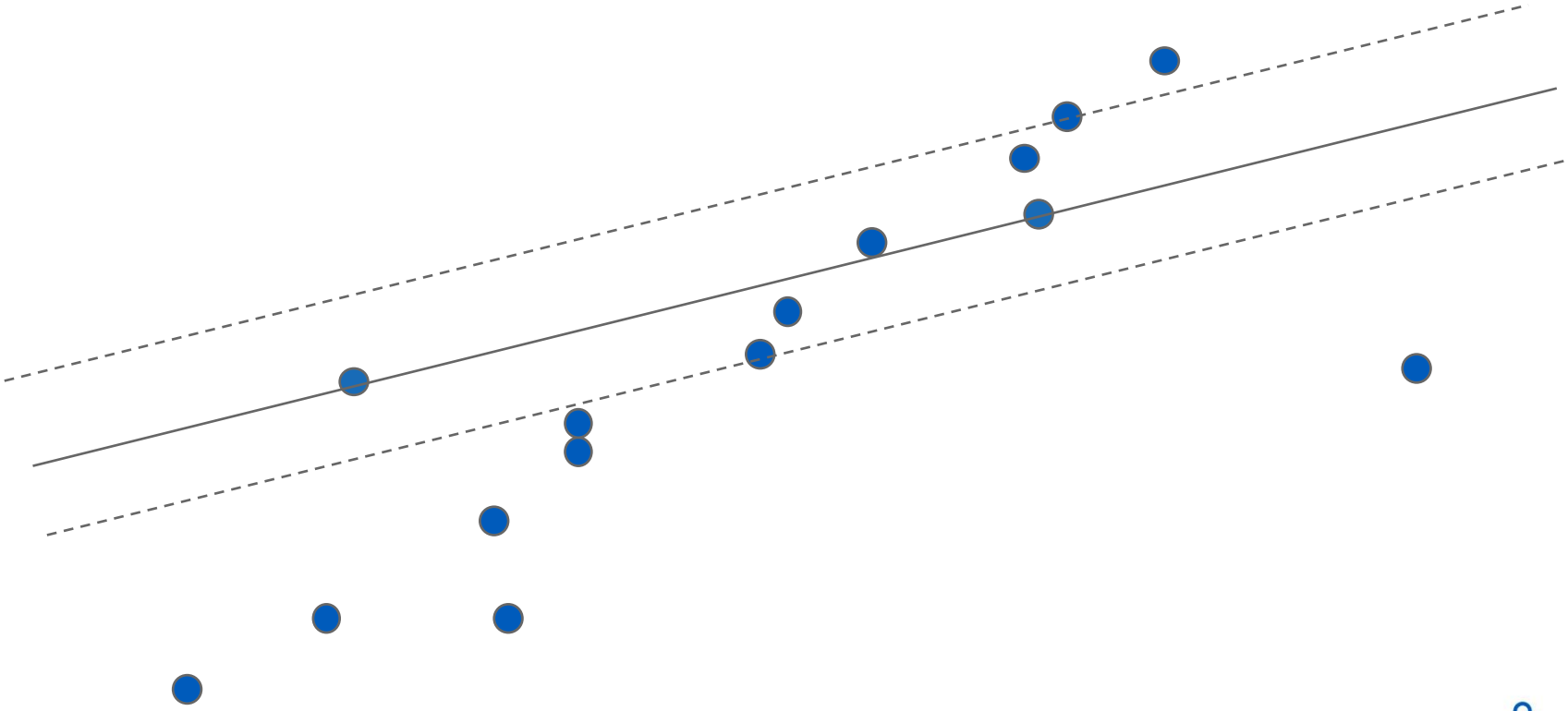
RANSAC Line Fitting Example

Fit Line



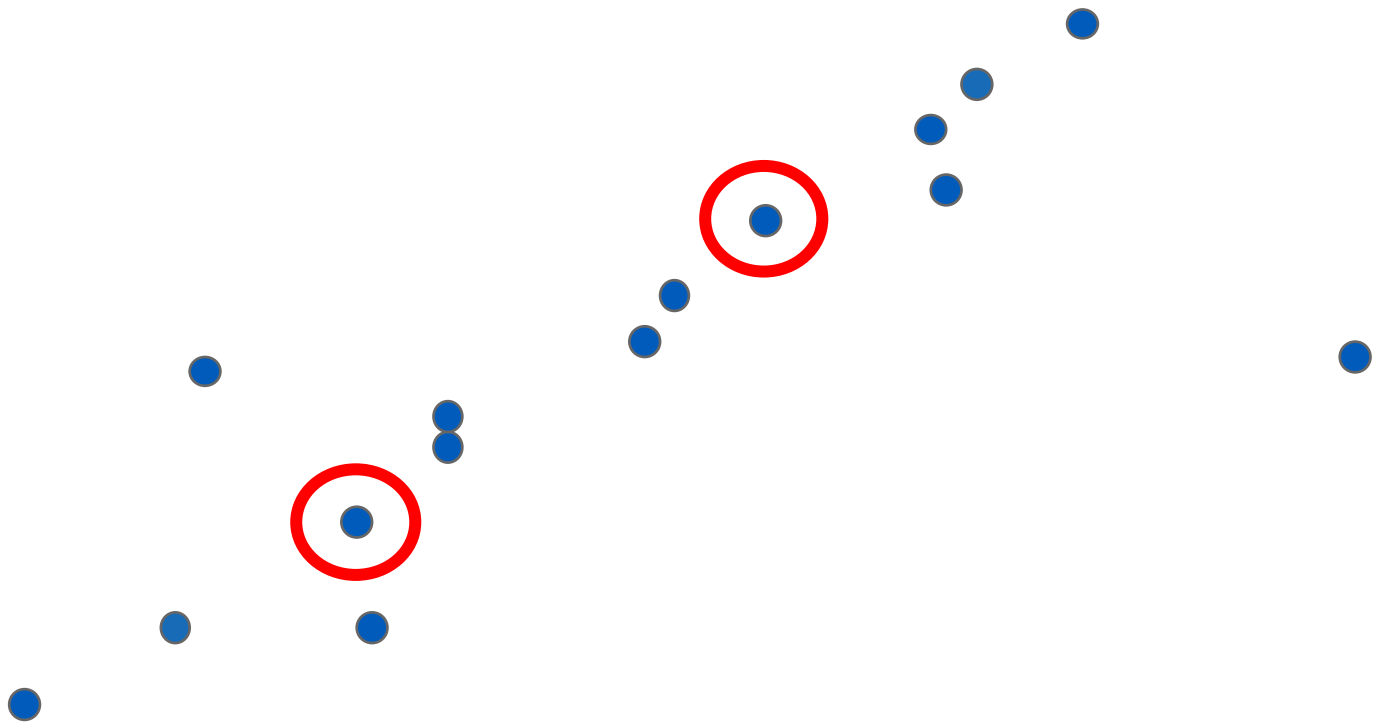
RANSAC Line Fitting Example

Total number of points within a threshold of line.



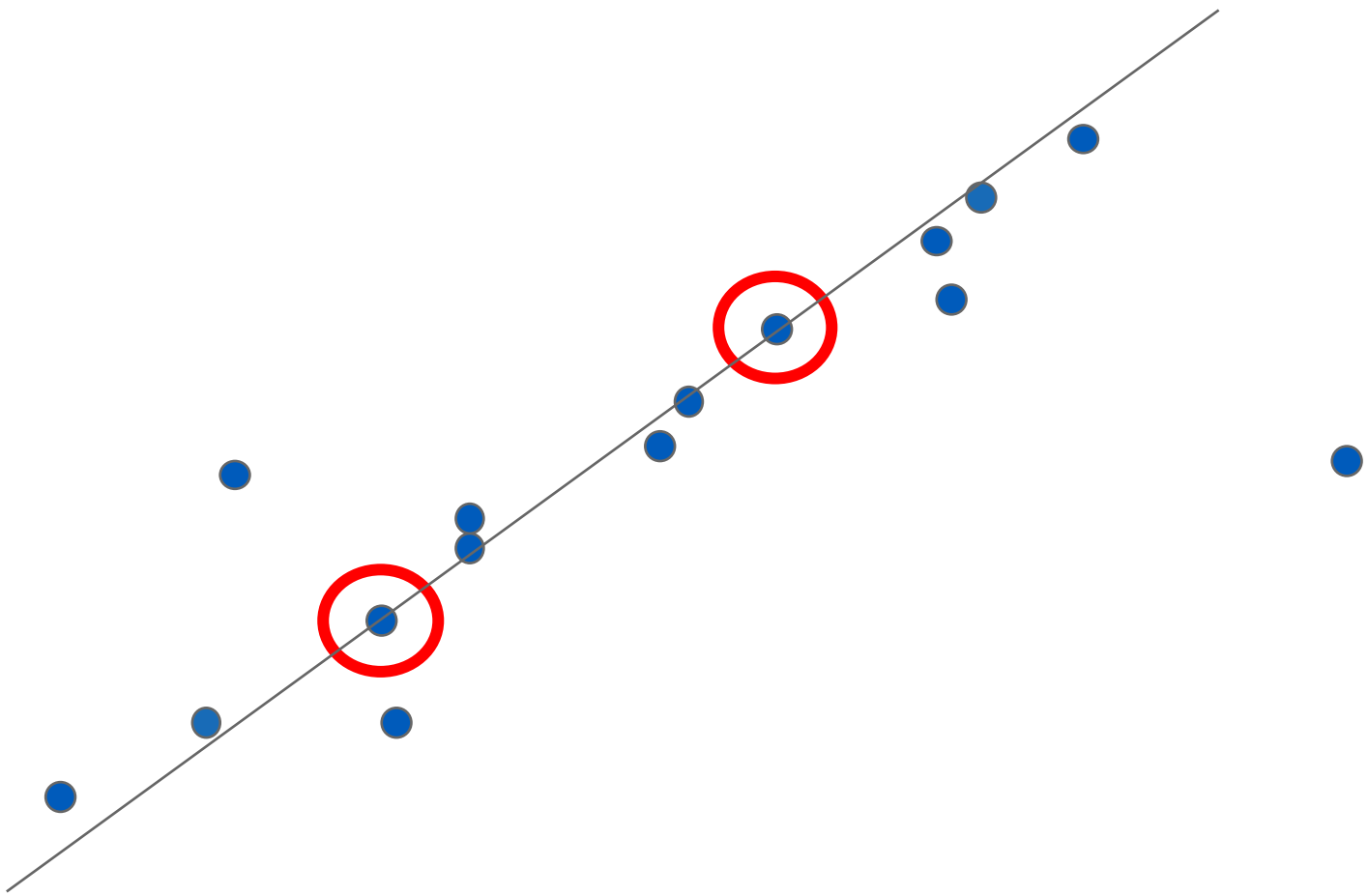
RANSAC Line Fitting Example

Repeat, until get a good result



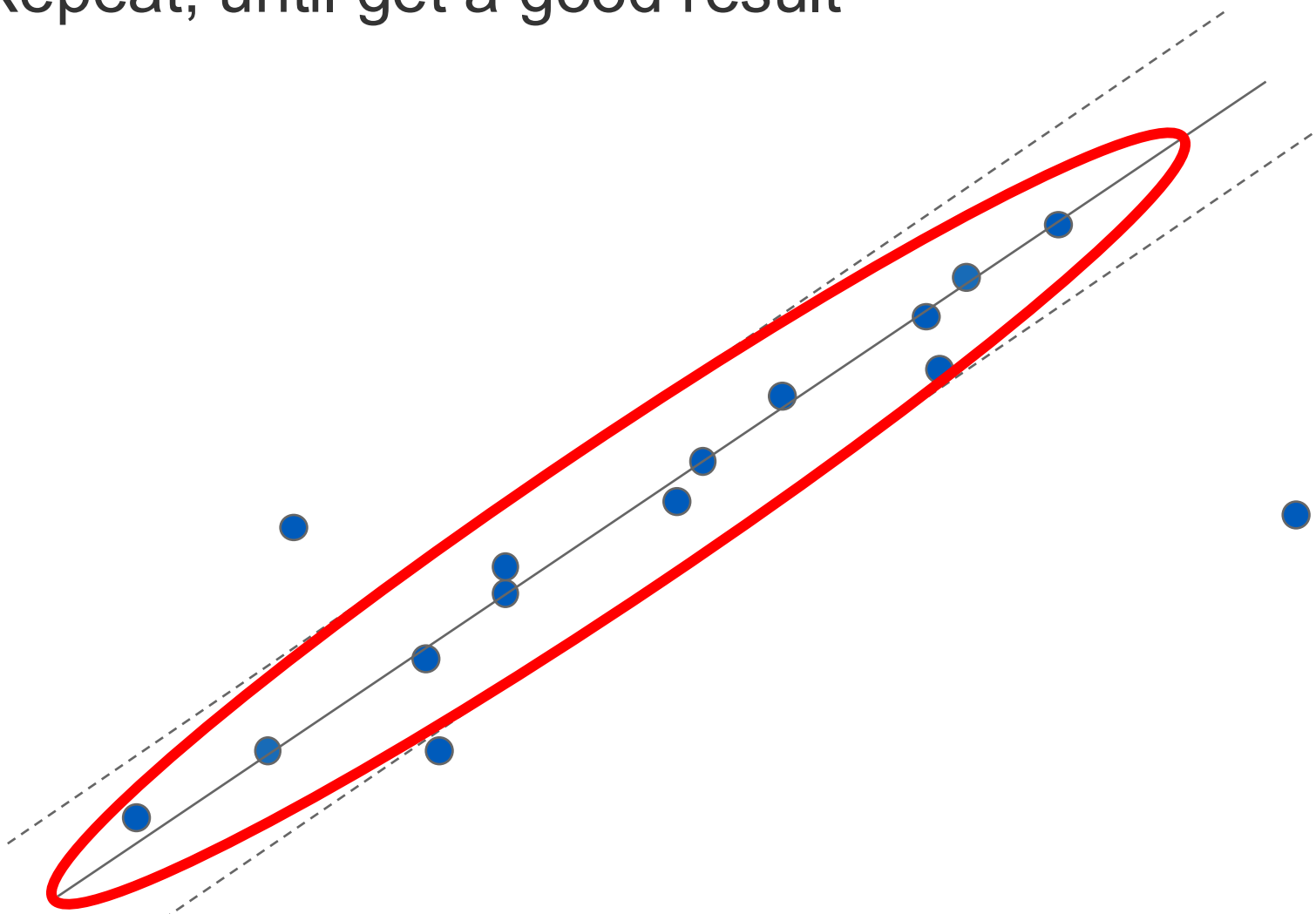
RANSAC Line Fitting Example

Repeat, until get a good result



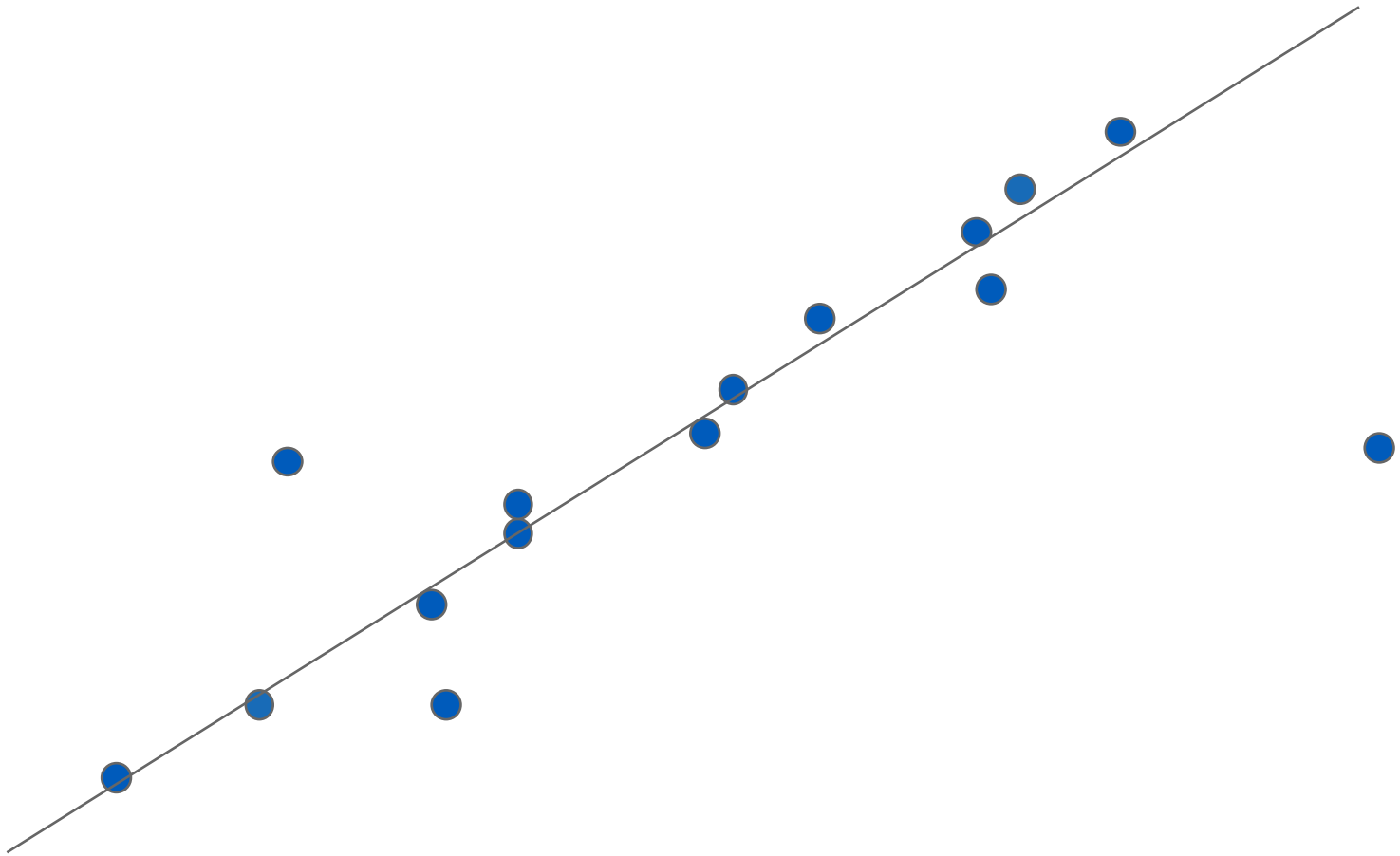
RANSAC Line Fitting Example

Repeat, until get a good result



RANSAC Line Fitting Example

Repeat, until get a good result

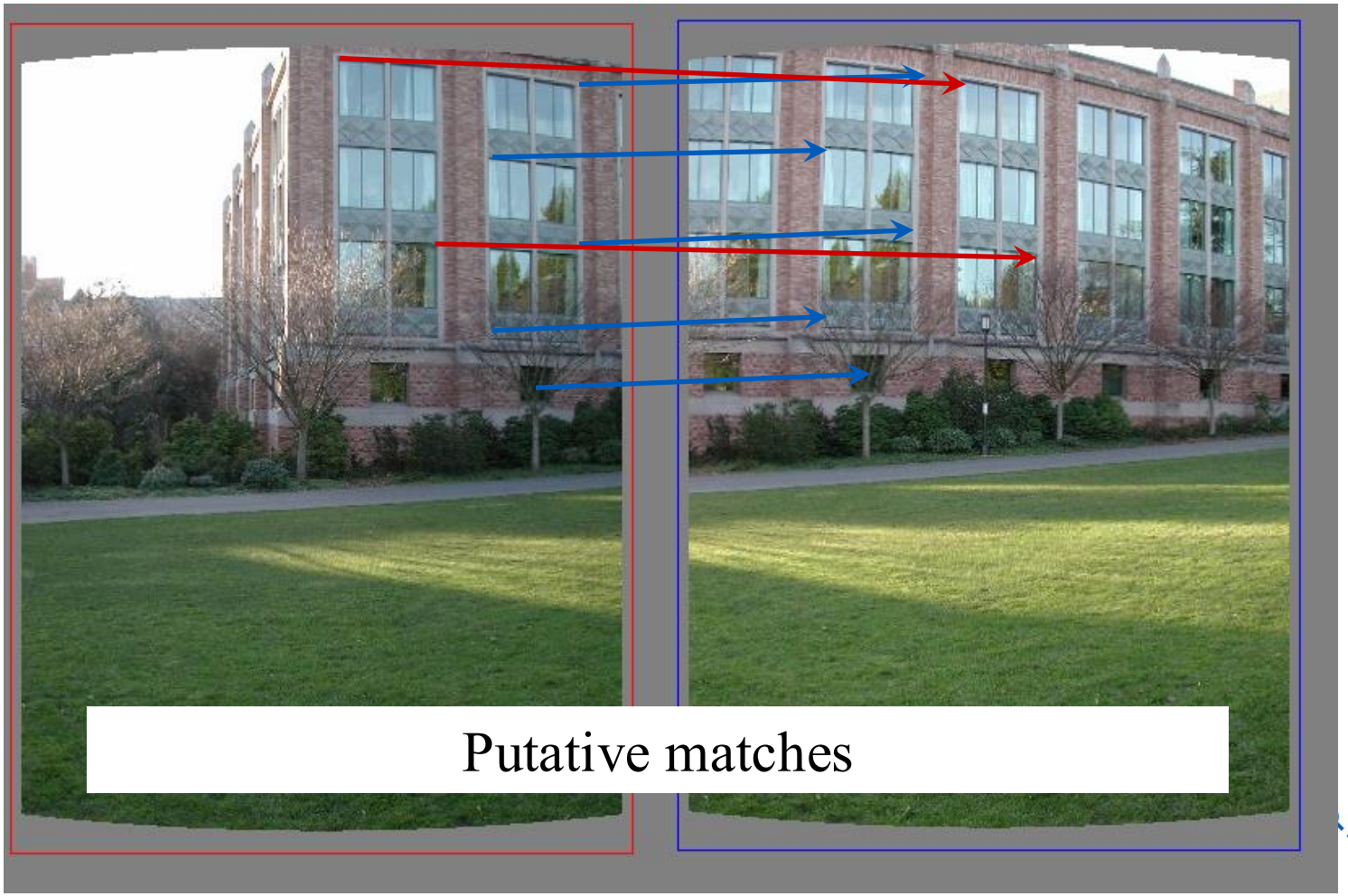


How to choose parameters?

- Number of sampled points n: minimum points to fit a model.
- Inlier threshold δ.
  - Choose δ so that **a good point with noise** is likely within threshold.
- To determine the number of iterations K.
  - Desired probability of success (p): at least one useful result.
  - Let w be the probability of choosing an inlier when selecting a point.
    - w = number of inliers in data / number of points in data
  - n points selected independently for estimating a model.
  - w^n: the probability that all n points are inliers.
  - $1 - w^n$: probability of at least one of the n points is an outlier.
  - $(1 - w^n)^K$: after k iterations, never select a set of n inlier points.
  - $1 - p = (1 - w^n)^K$
  - $K = \frac{\log(1-p)}{\log(1-w^n)}$

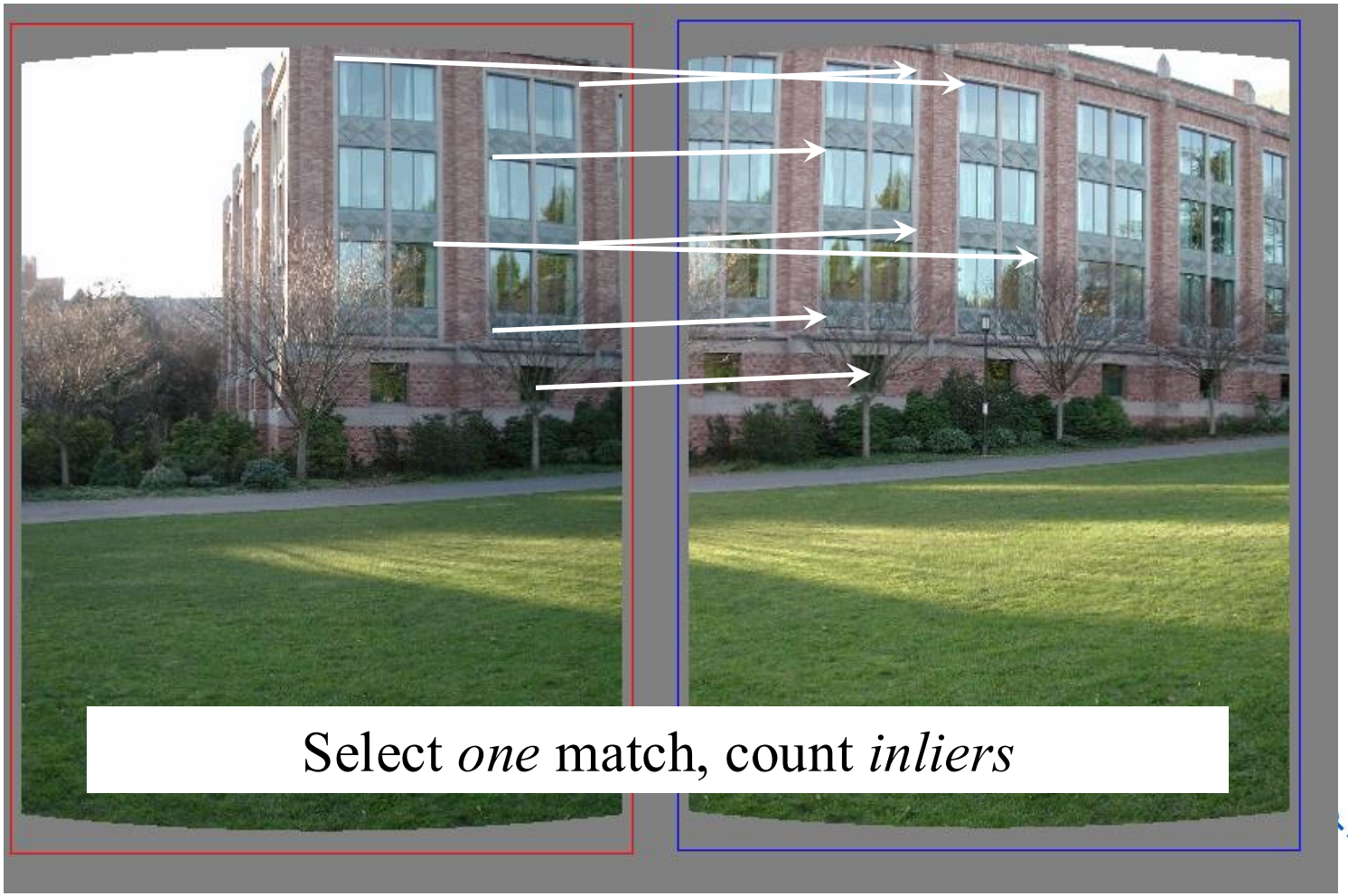
	$p = 0.99$, proportion of outliers $(1 - w)$						
n	$1 - w = 5\%$	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

RANSAC example: Translation

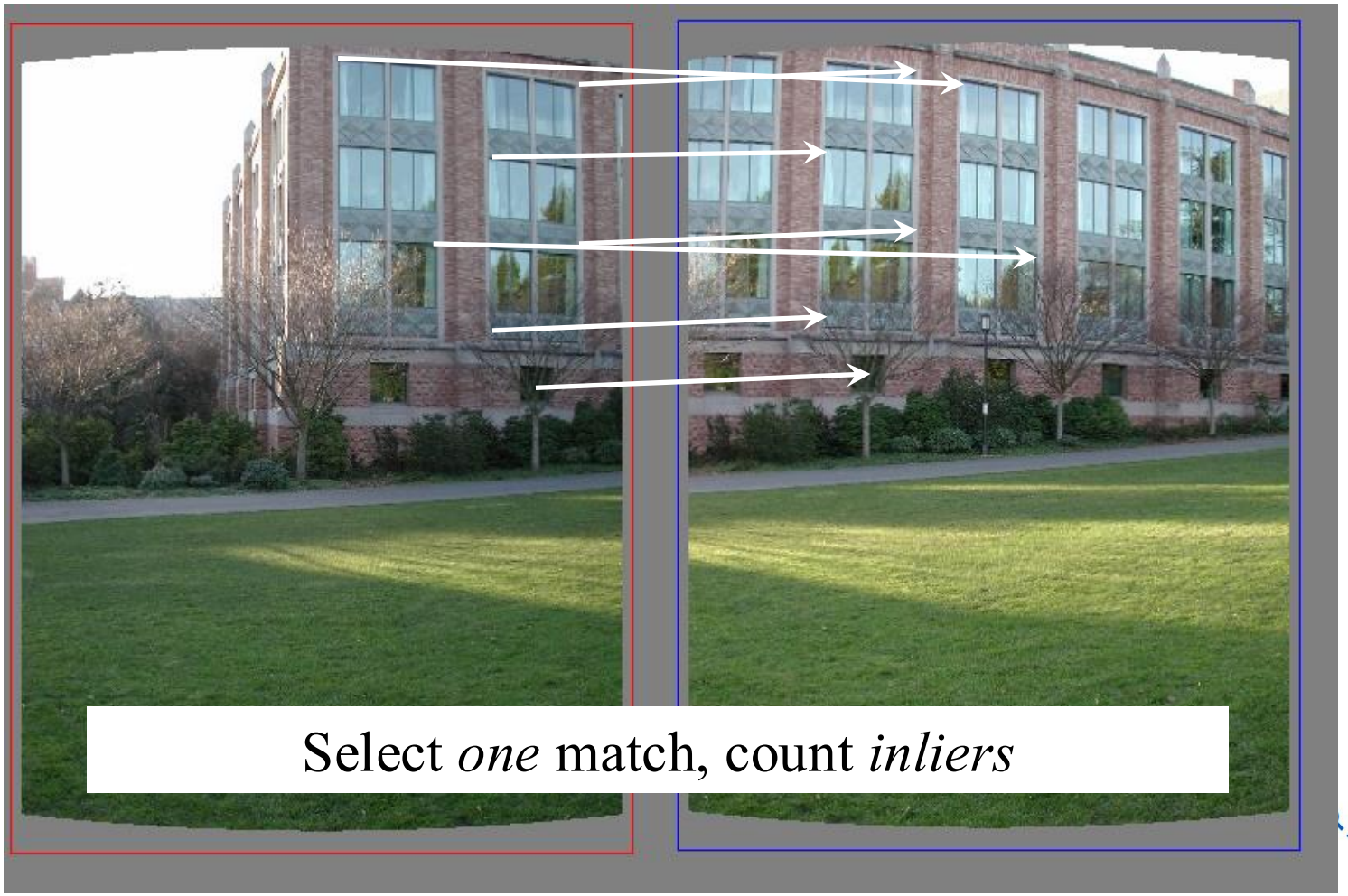


Source: Rick Szeliski

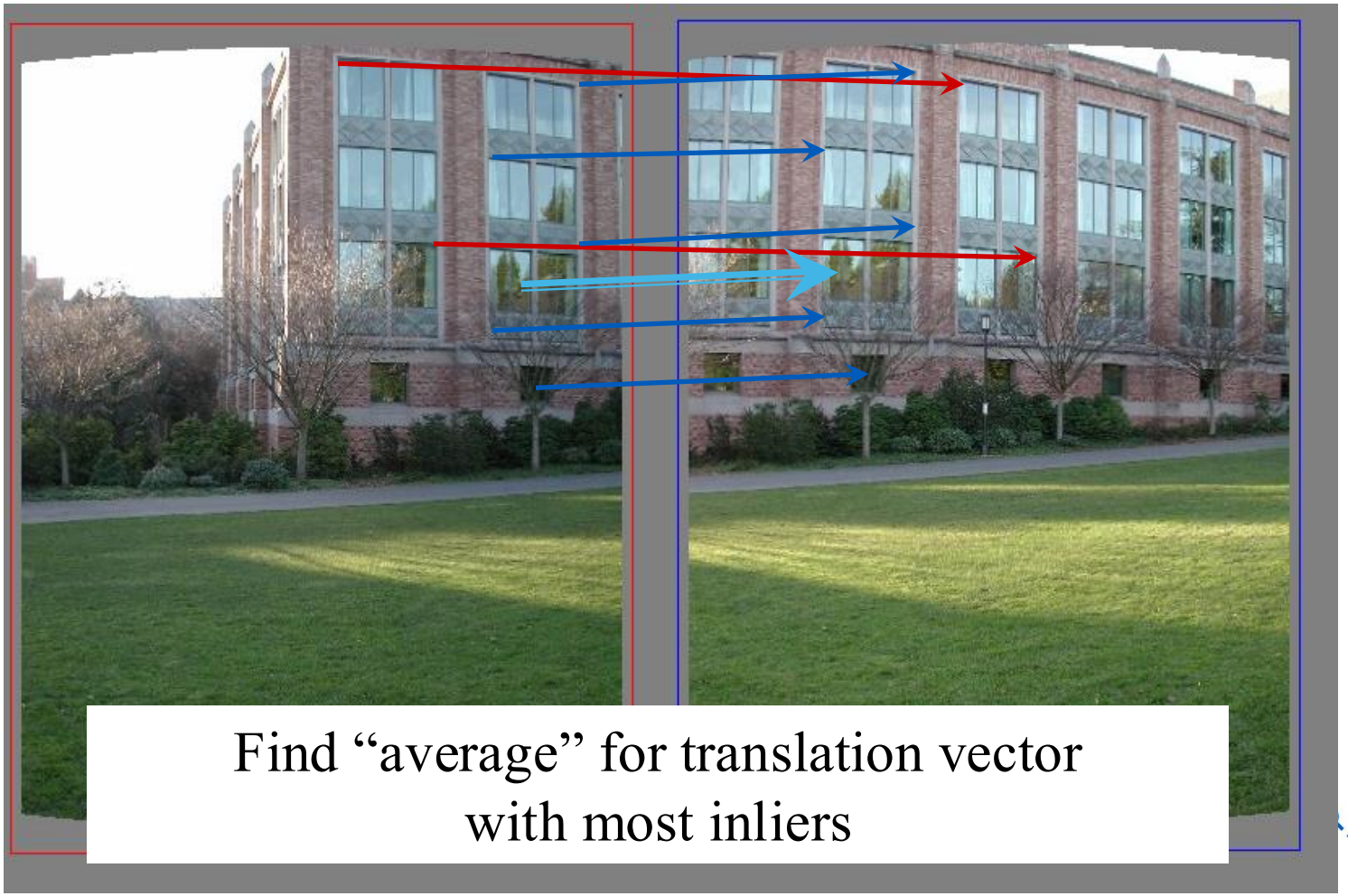
RANSAC example: Translation



RANSAC example: Translation



RANSAC example: Translation



Find “average” for translation vector
with most inliers

Summary

- Choose:
  - Inlier threshold
    - Related to the amount of noise we expect
  - Number of rounds
    - Related to the percentage of outliners we expect
    - Related to the probability of success we are hoping for.

RANSAC ALGORITHM

- Input:
  - Data: a set of observed data points
  - Model: a model to fit the data
  - Threshold: a threshold to determine inliers
- Output: BestModel: the model with the most inliers
- Repeat for a fixed number of iterations:
  - 1. Select a random subset of data
  - 2. Fit the model to the data points in the subset
  - 3. Determine the inliers by comparing the fitted model to data
  - 4. If the number of inliers exceeds the threshold
    - re-estimate the model using all the inliers
  - 5. Store the model if it has the most inliers seen so far
- Return BestModel

RANSAC Properties

Strength

- Robust to outliers
- Applicable for larger number of model parameters than Hough transform.
- Optimization parameters are easier to choose than Hough transform.
  - Lines with normal form works for Hough, but slope-intercept form not.

Weakness

- Computational time grows quickly with outliers and parameters
  - While Hough transform grows quickly with number of parameters.
- Not good for getting multiple fits
  - Hough transform can fit multiple lines simultaneously.

More applications

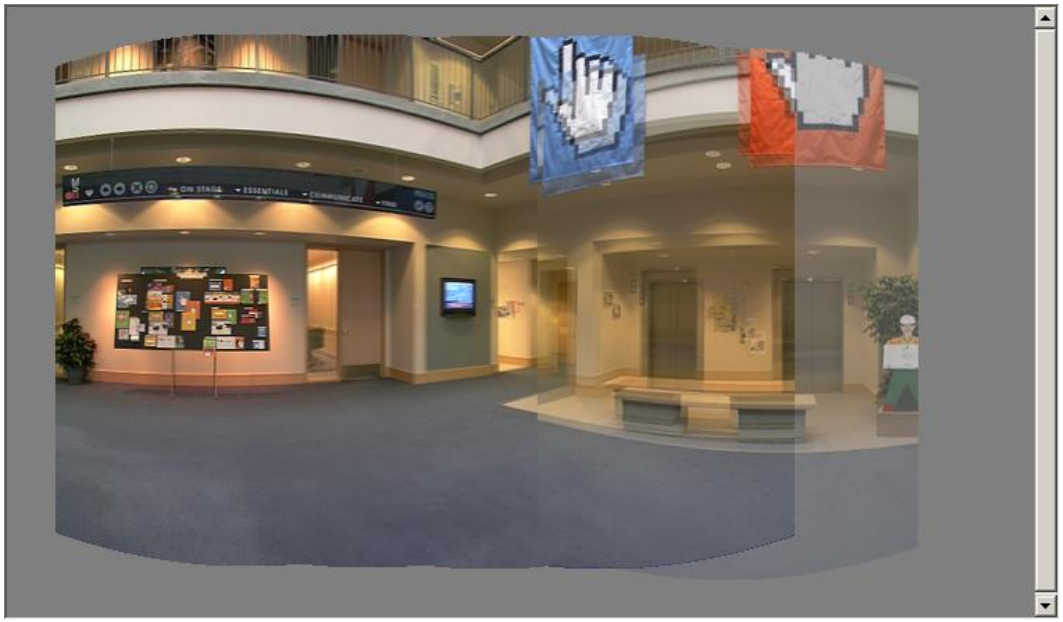
- Computing a homography (e.g., image stitching)
- Estimating fundamental matrix (relating two views)

Content

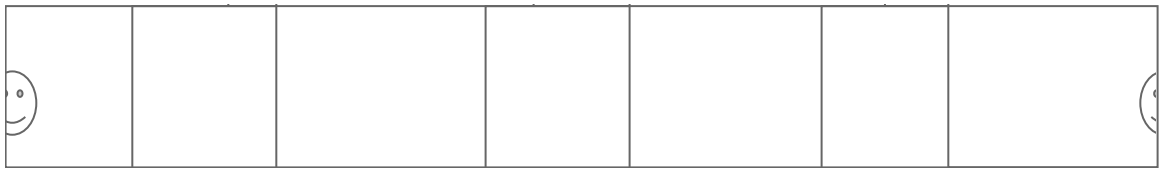
- Stitching (Blending)
  - Panorama, Blending, Deghosting
  - Feathering, Pyramid Blending
  - Laplacian Blending

Image Stitching

1. Align the images over each other
   - camera pan ↔ translation on cylinder
2. Blend the images together

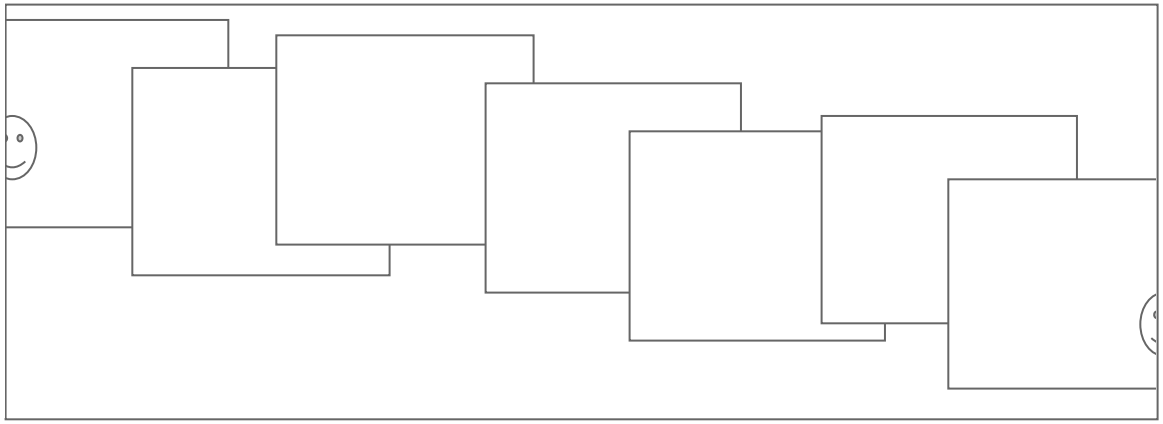


Assembling the panorama



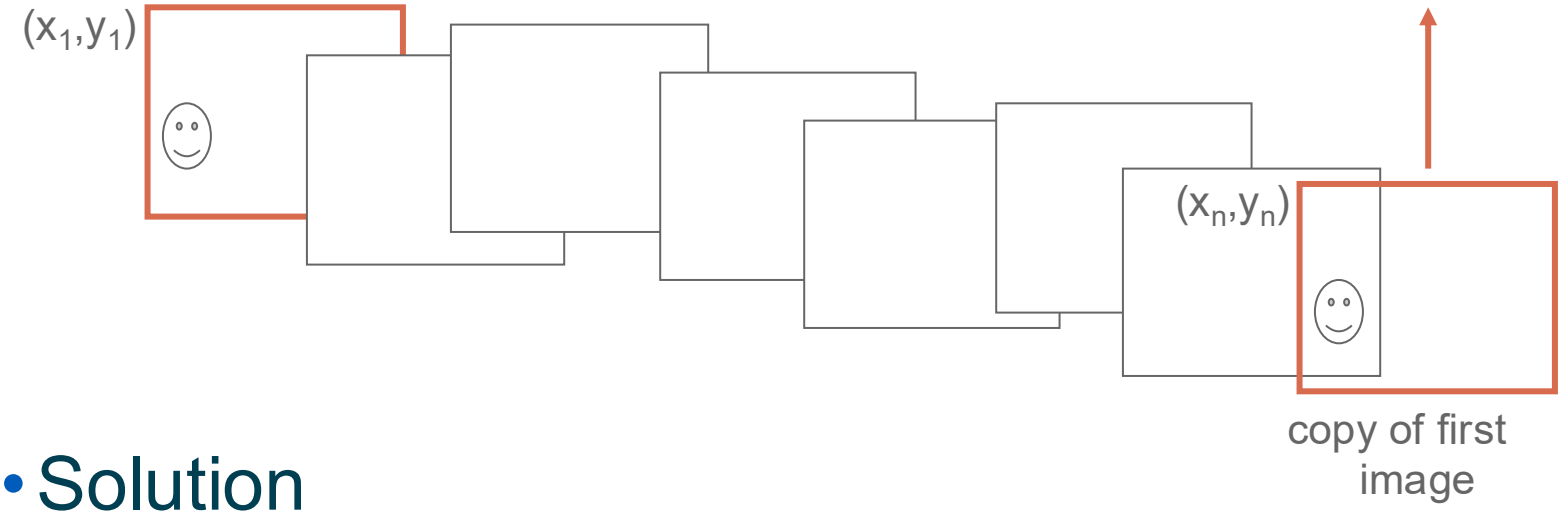
- Stitch pairs together, blend, then crop

Problem: Drift



- Error accumulation
  - small (vertical) errors accumulate over time
  - apply correction so that sum = 0 (for 360° pan.)

Problem: Drift



- Solution
  - add another copy of first image at the end
  - this gives a constraint: $y_n = y_1$
  - there are a bunch of ways to solve this problem
    - add displacement of $(y_1 – y_n)/(n -1)$ to each image after the first
    - compute a global warp: $y' = y + ax$
    - run a big optimization problem, incorporating this constraint
      - best solution, but more complicated.

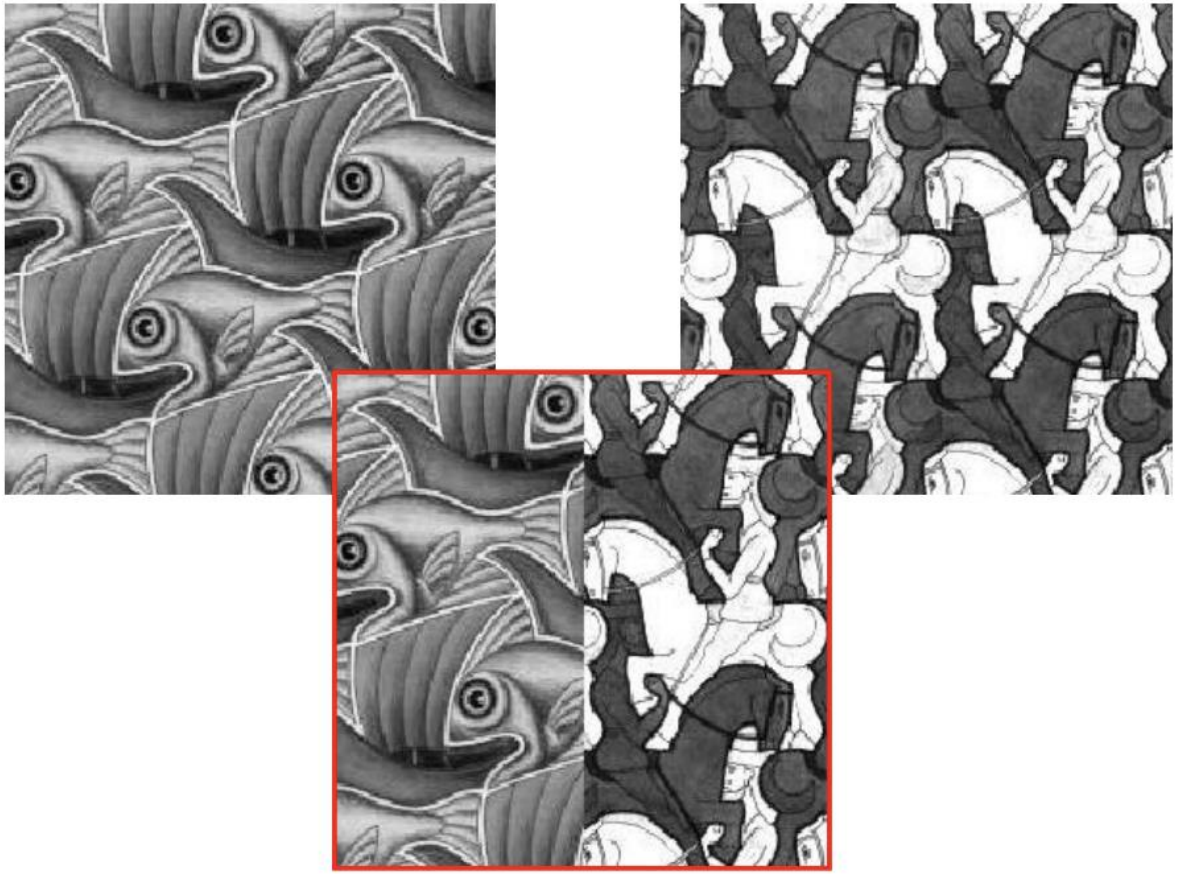
Full-view (360º spherical) panoramas

Full-view Panorama

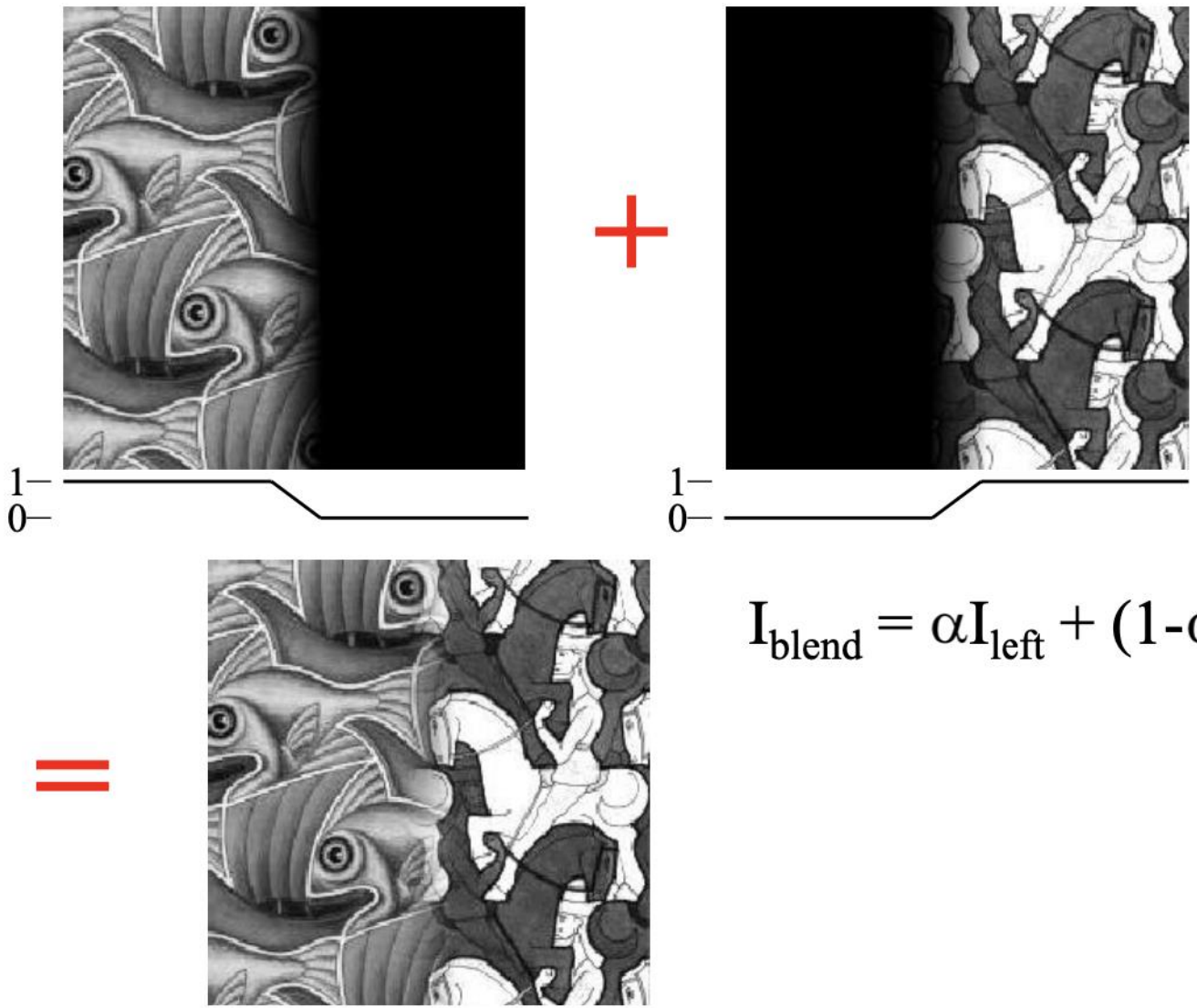
Texture Mapped Model

Image Blending

- How do we put images together so there is no obvious divisions?

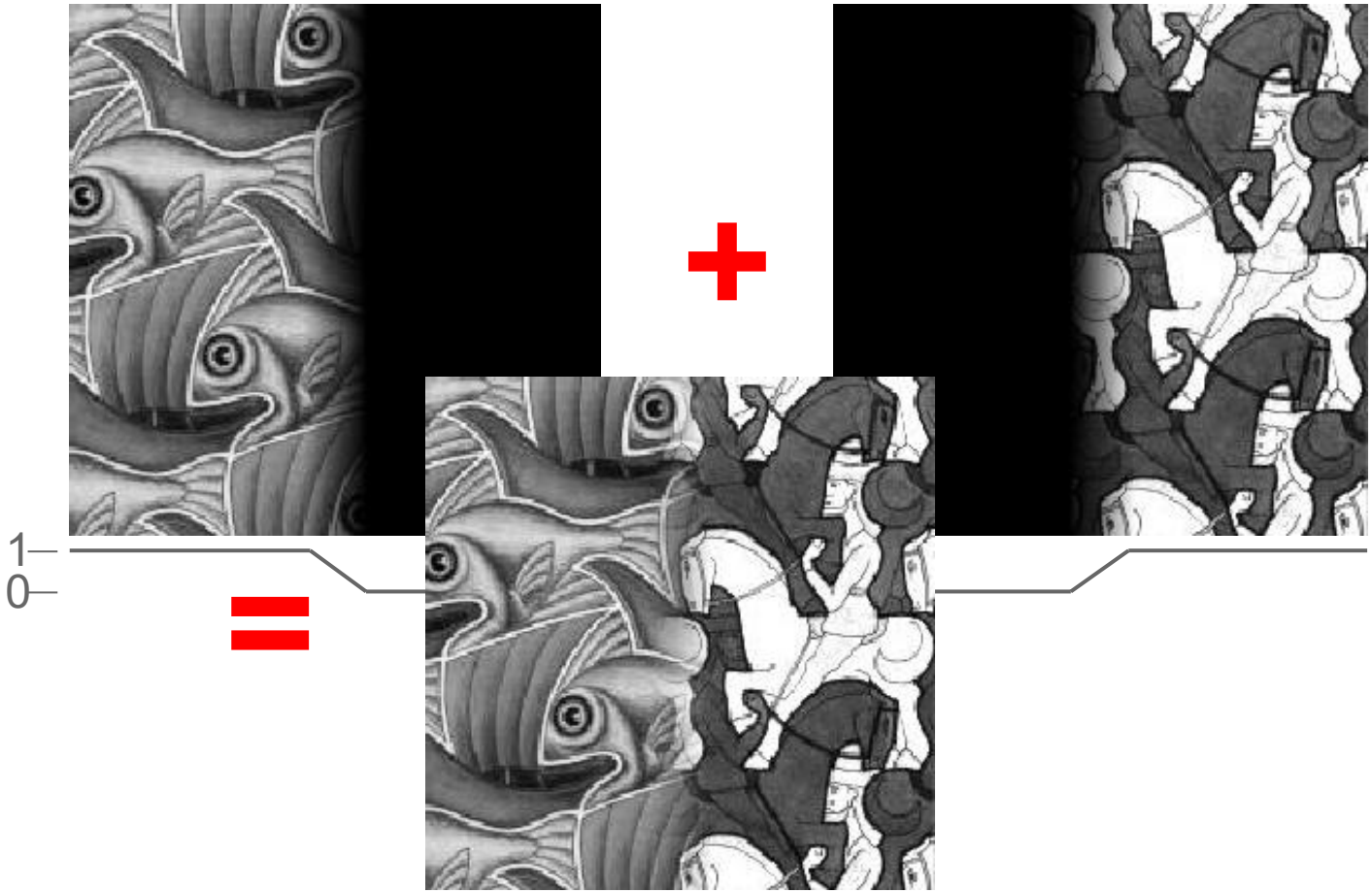


Alpha Blending / Feathering

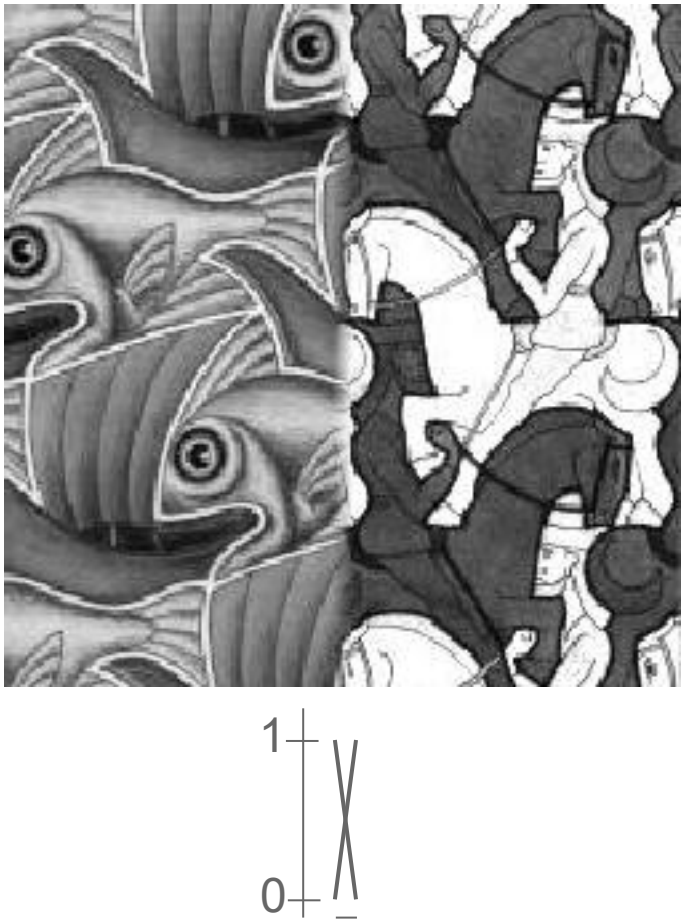


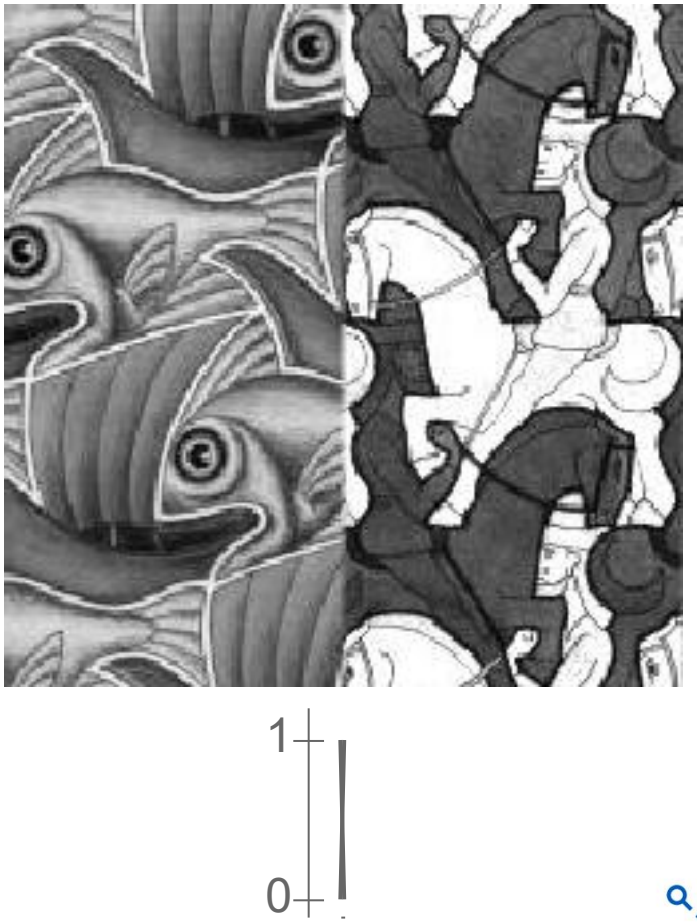
$$I_{blend} = \alpha I_{left} + (1-\alpha) I_{right}$$

Feathering

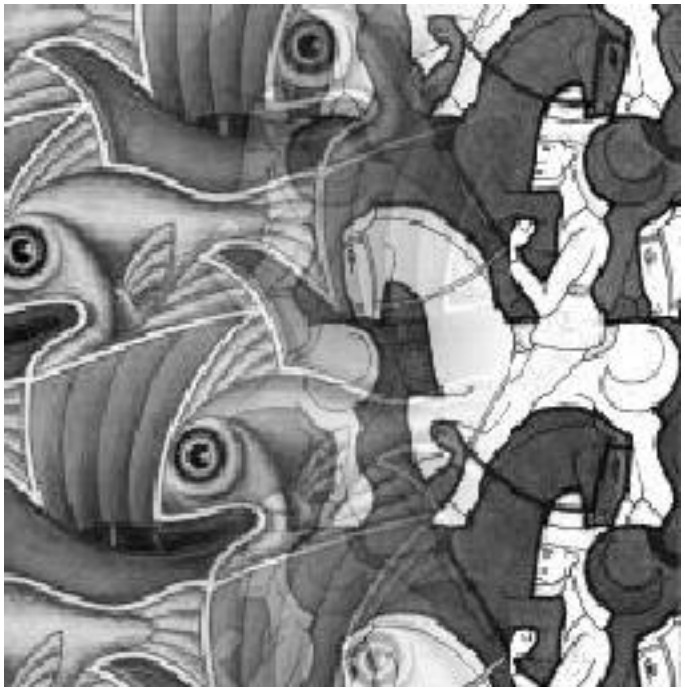


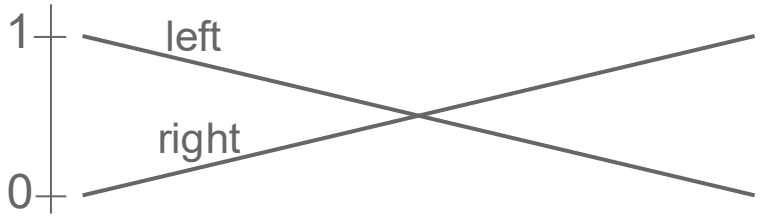
Effect of window size

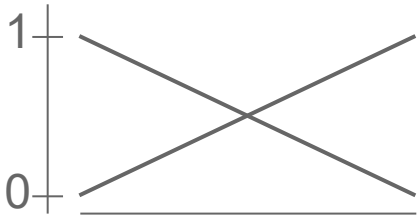




Effect of window size

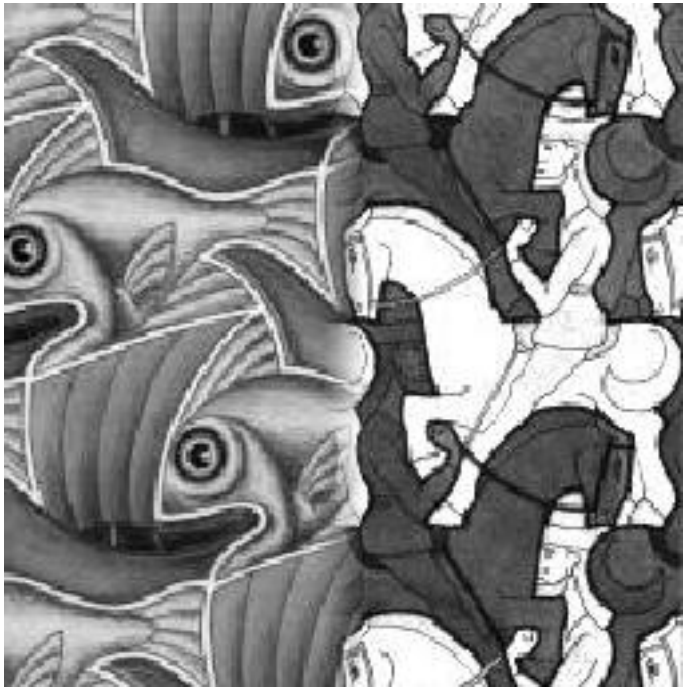






Good Window Size

- “Optimal” Window: smooth but not ghosted.
- To avoid seams
  - window = size of largest prominent feature
- To avoid ghosting
  - window <= 2*size of smallest prominent feature

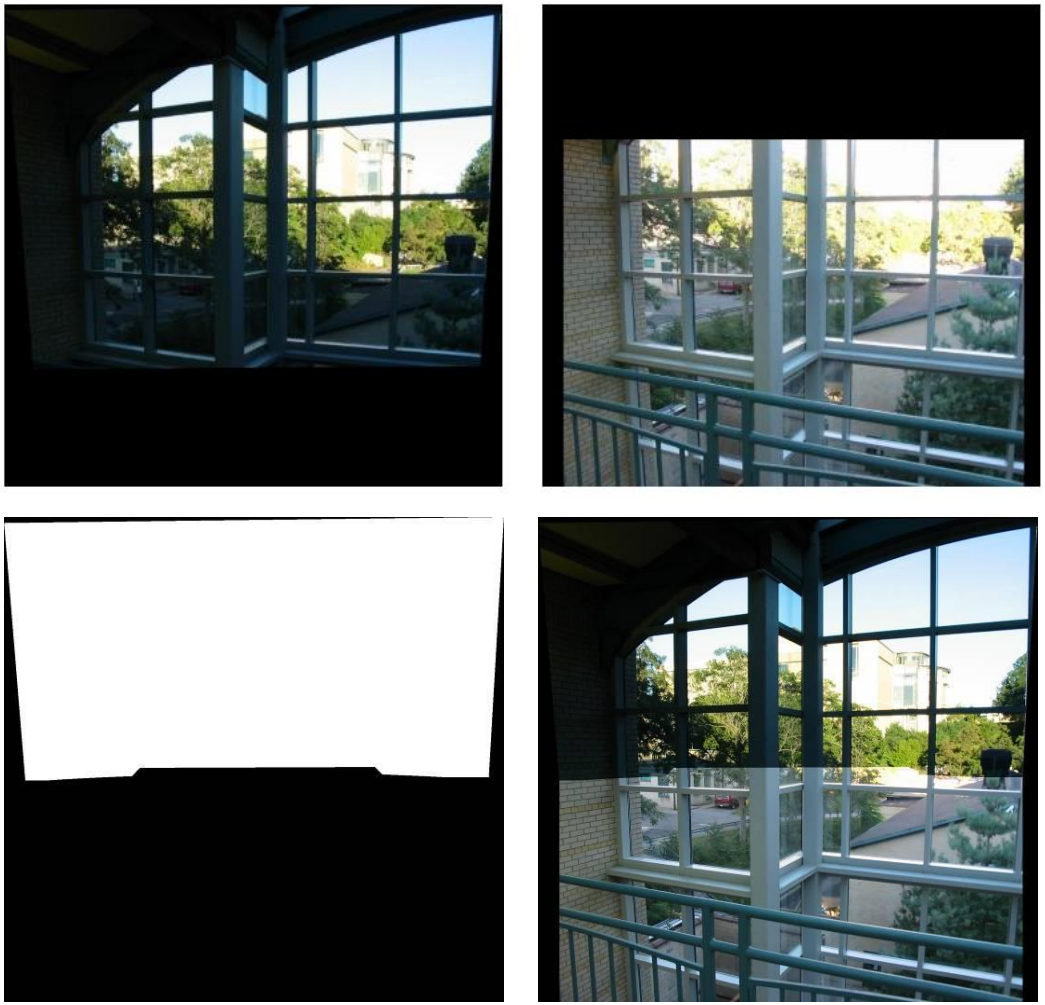


Setting alpha: simple averaging

- Alpha = 0.5 in the overlap region

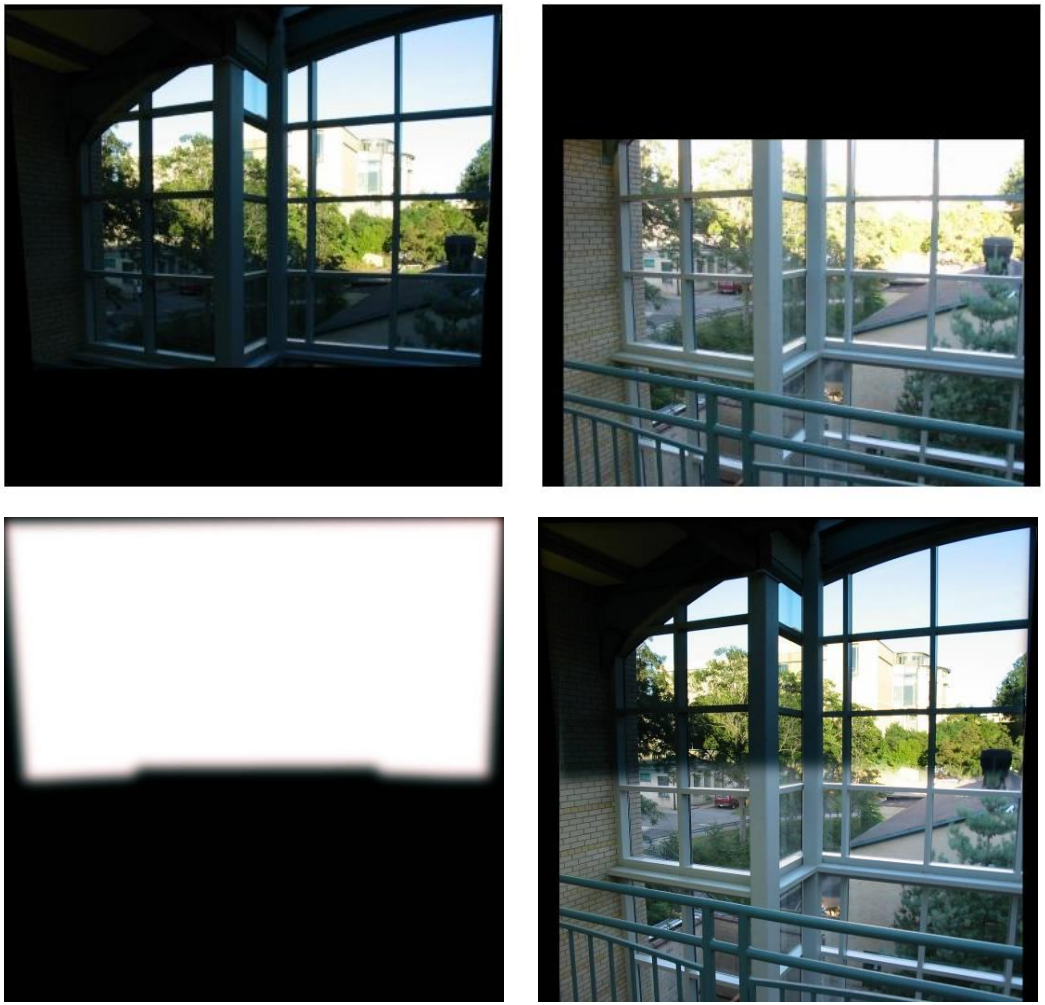
Setting alpha: center seam

- Alpha = logical(dtrans1>dtrans2)



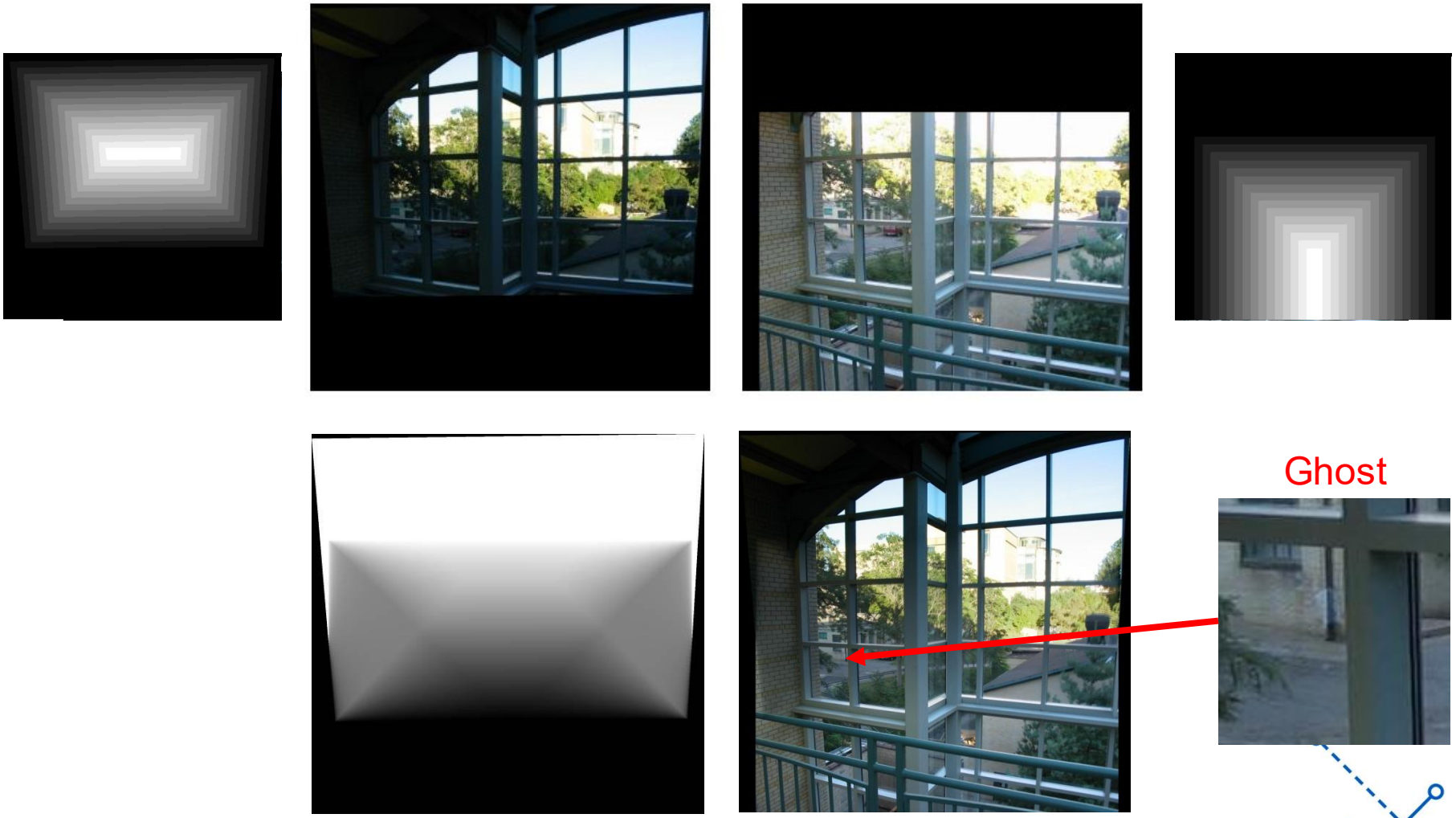
Setting alpha: blurred seam

- Alpha = blurred



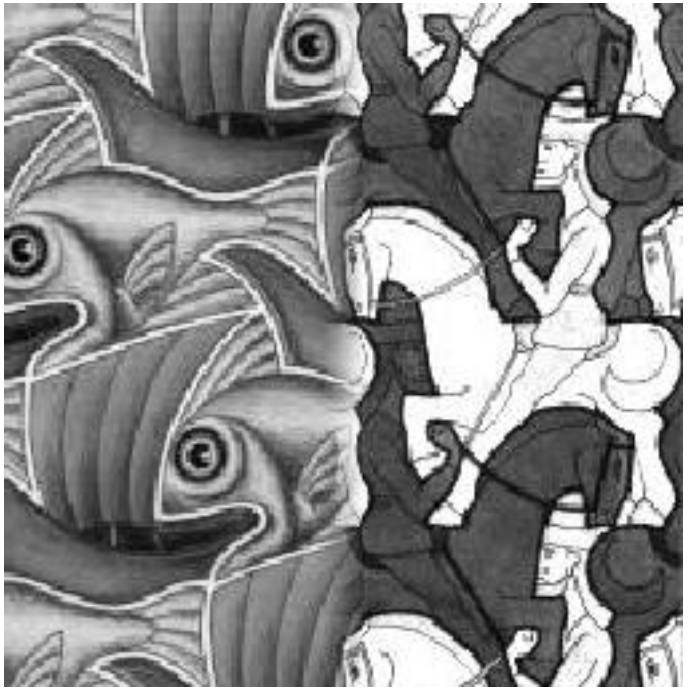
Setting alpha: center weighting

- Alpha = dtrans1 / (dtrans1+dtrans2)

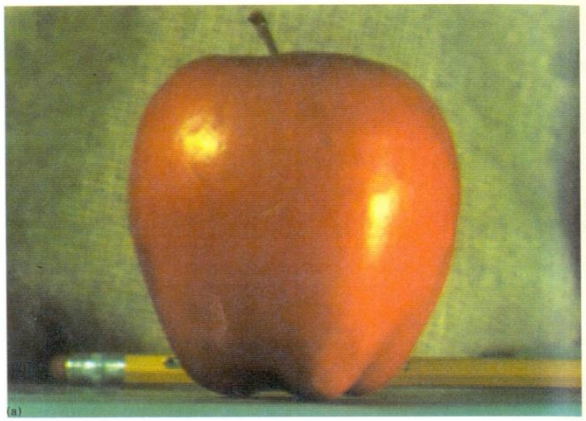


Recap: Good Window Size

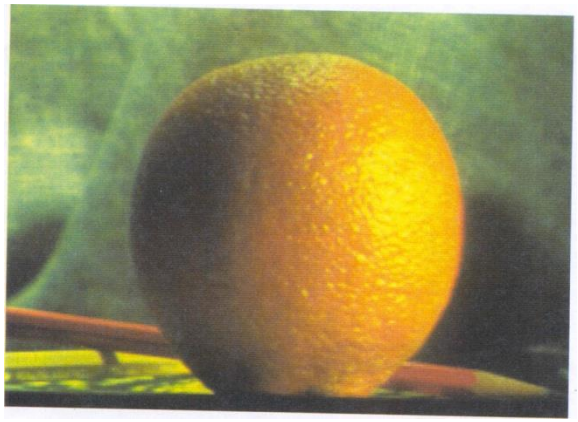
- “Optimal” Window: smooth but not ghosted.
- To avoid seams
  - window = size of largest prominent feature
- To avoid ghosting
  - window <= 2*size of smallest prominent feature

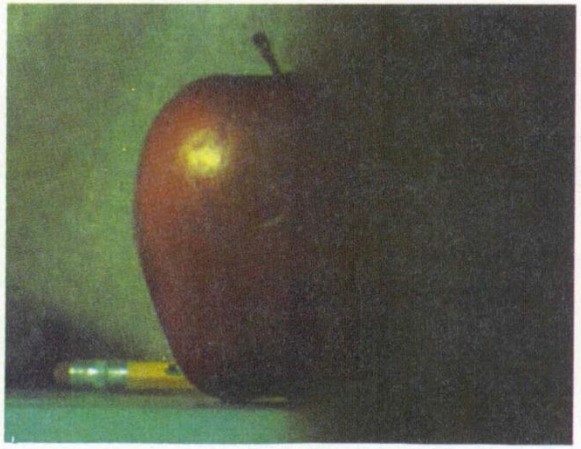


Pyramid Blending

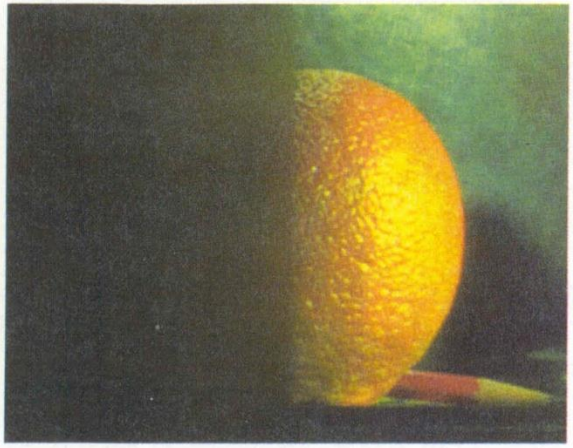


(a)

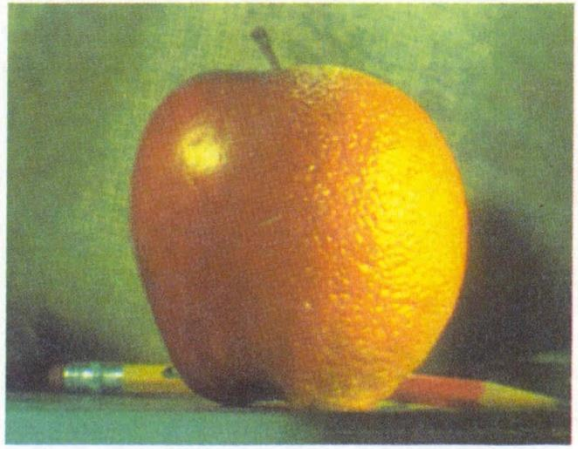




(d)



(h)



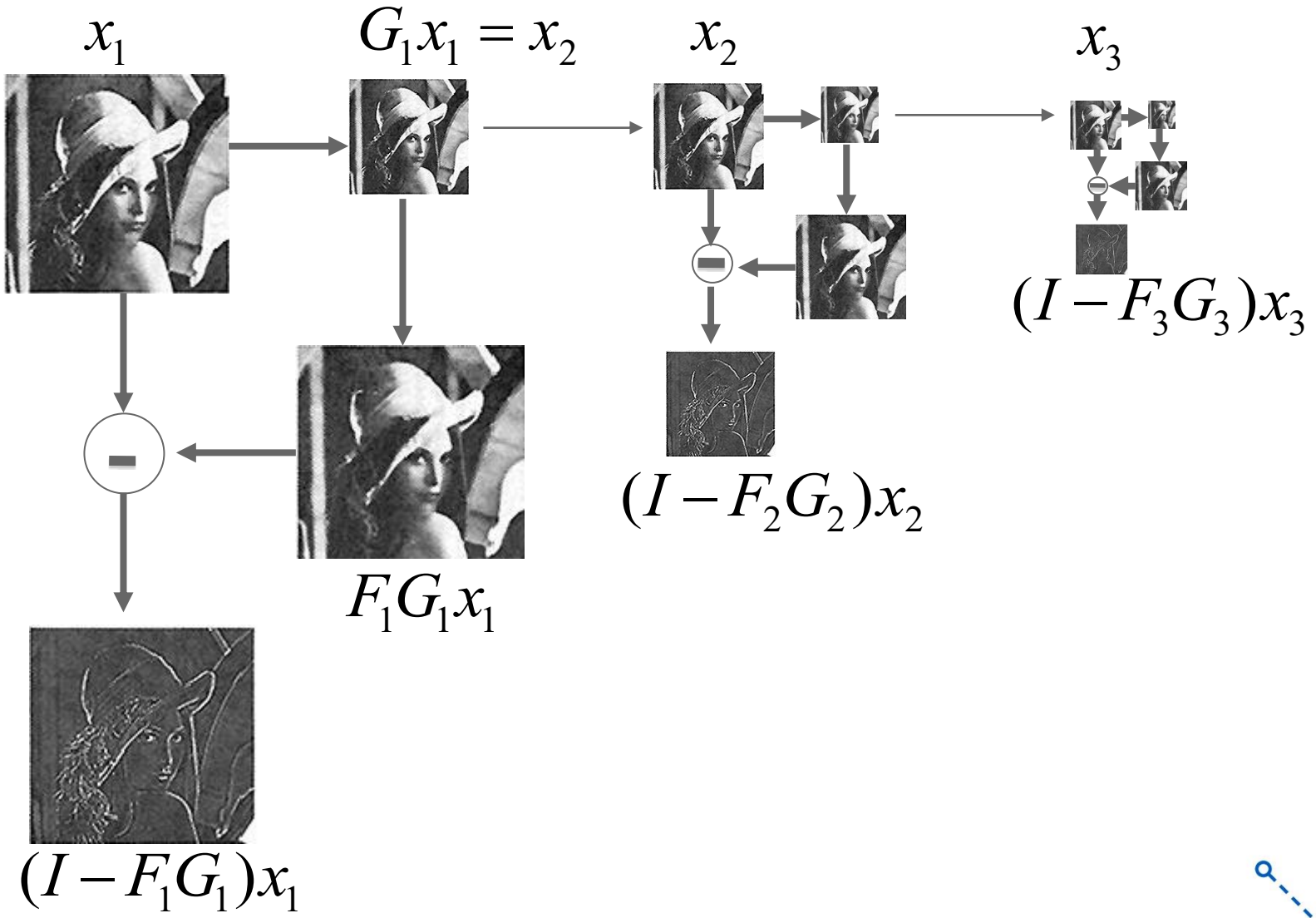
(l)

Burt, P. J. and Adelson, E. H., A multiresolution spline with applications to image mosaics, ACM Transactions on Graphics, 42(4), October 1983, 217-236.

Laplacian Pyramid (Recap)

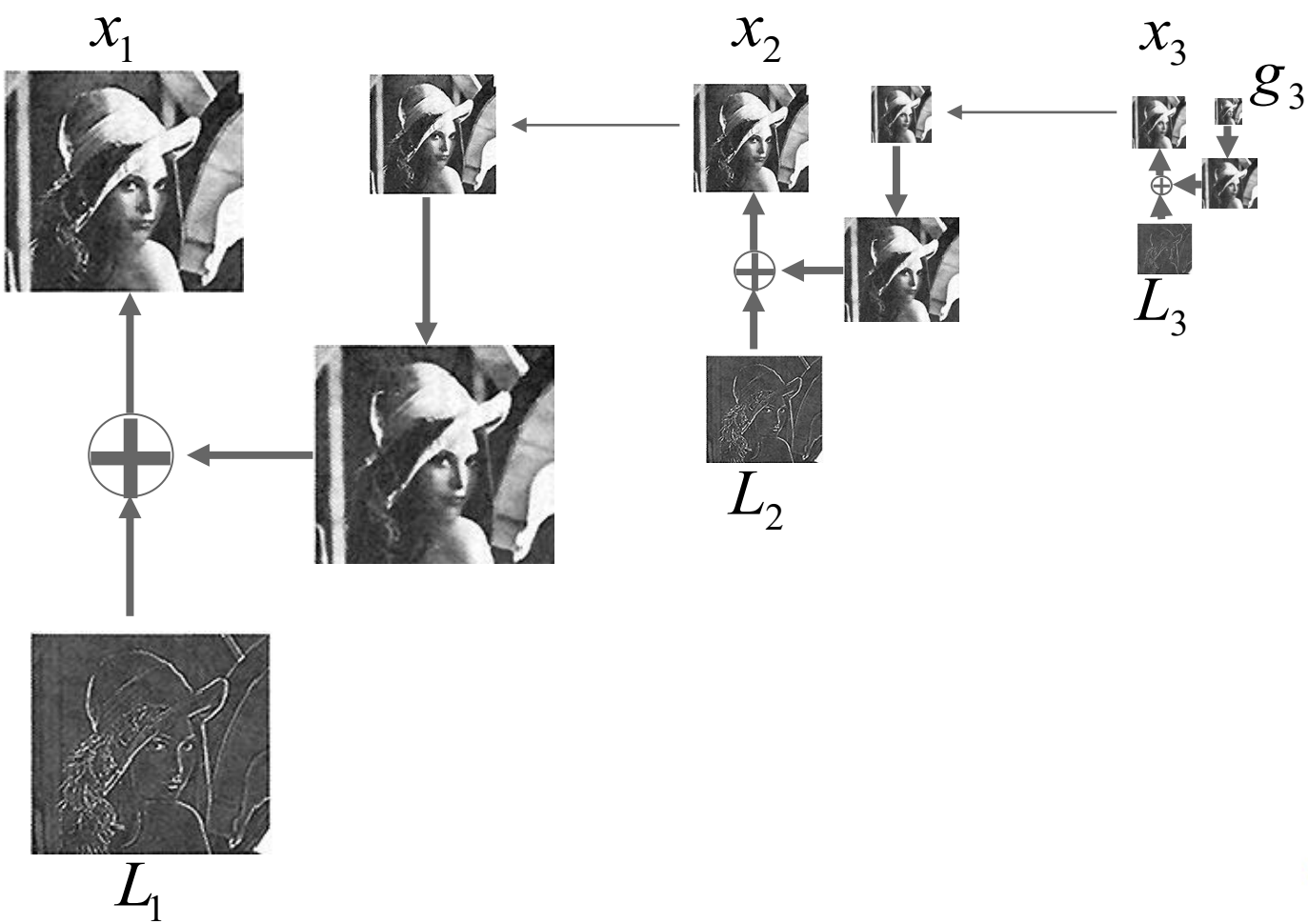
- **Difference between up-sampled Gaussian pyramid and Gaussian pyramid**.
- **Band pass filter** - each level represents spatial frequencies (largely) unrepresented at other level.

Laplacian pyramid (Recap)



Laplacian Pyramid (Recap)

- Reconstruction: recover x_1 from L_1, L_2, L_3 and g_3



Laplacian Pyramid (Recap)

- Information captured at each level of a Gaussian (top) and Laplacian (bottom) pyramid
  - showing full resolution.

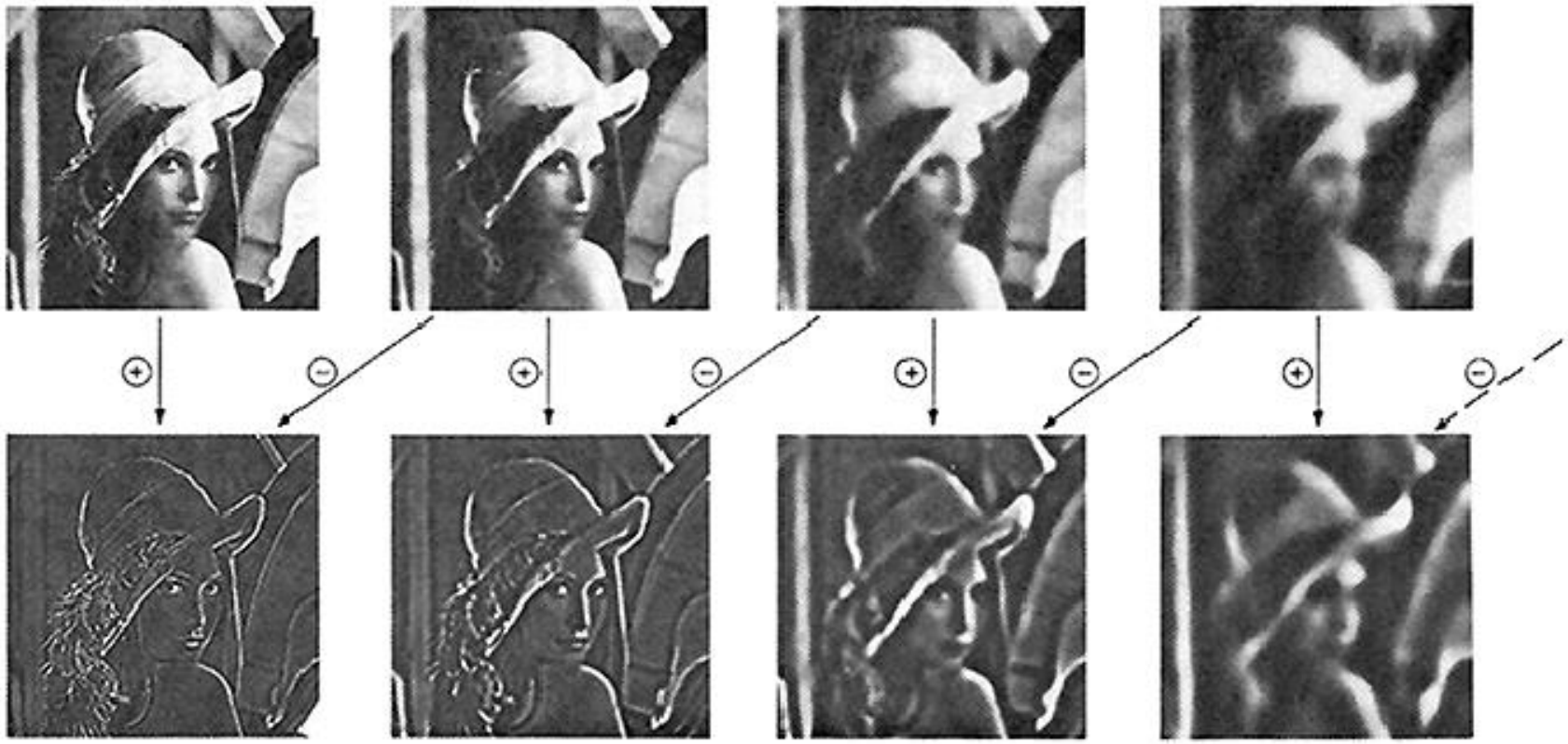
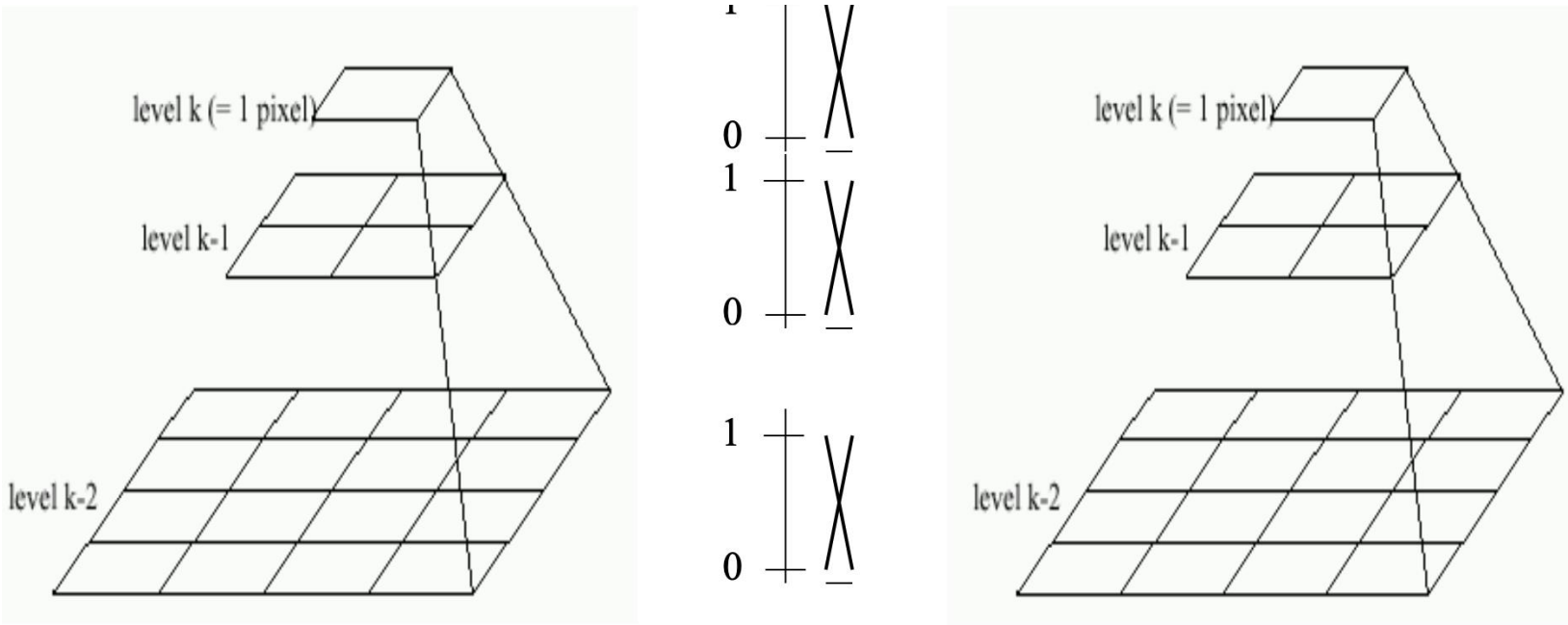


Fig 5. First four levels of the Gaussian and Laplacian pyramid. Gaussian images, upper row, were obtained by expanding pyramid arrays (Fig. 4) through Gaussian interpolation. Each level of the Laplacian pyramid is the difference between the corresponding and next higher levels of the Gaussian pyramid.

Pyramid Blending

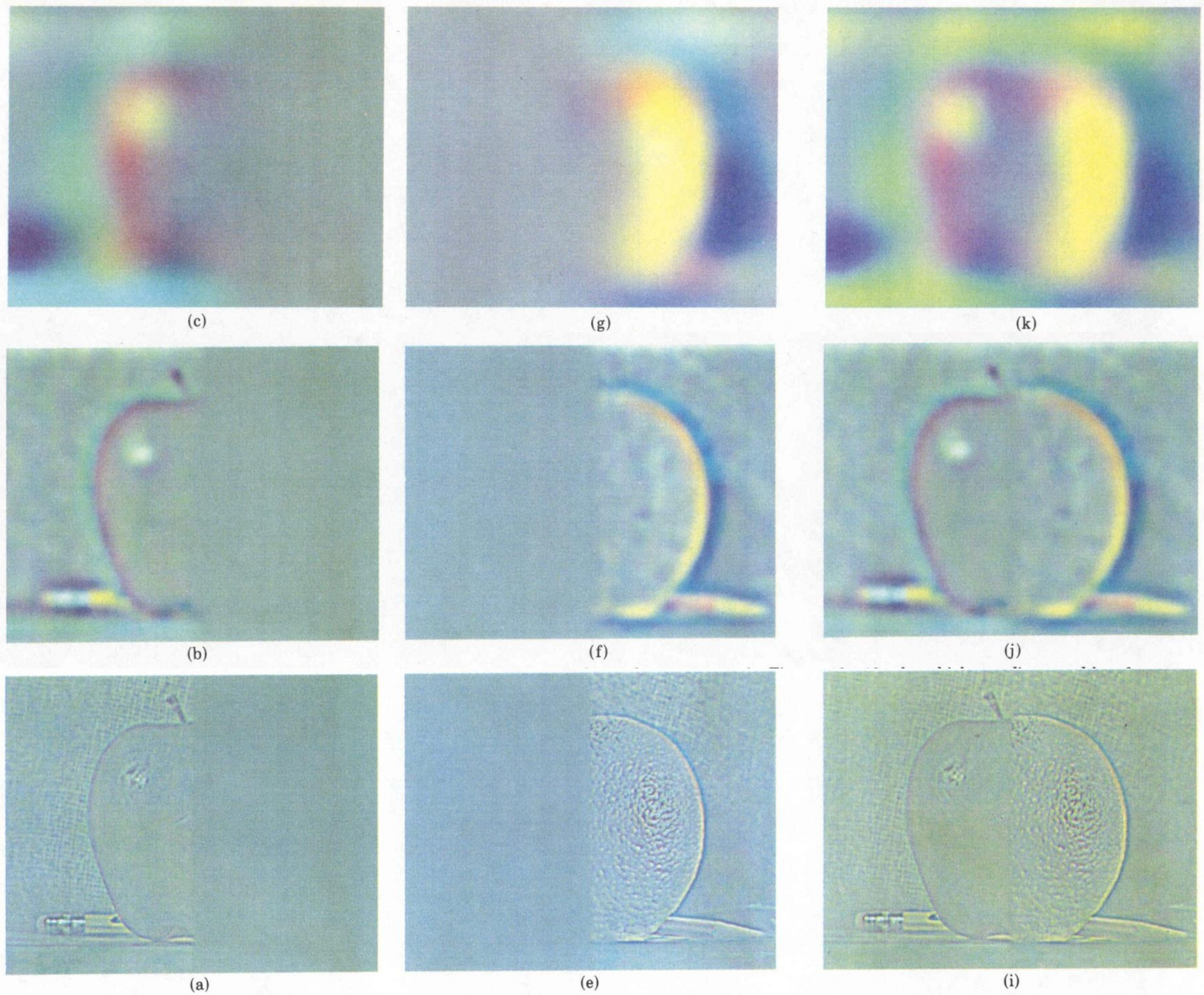


Left pyramid blend Right pyramid

Laplacian level 4

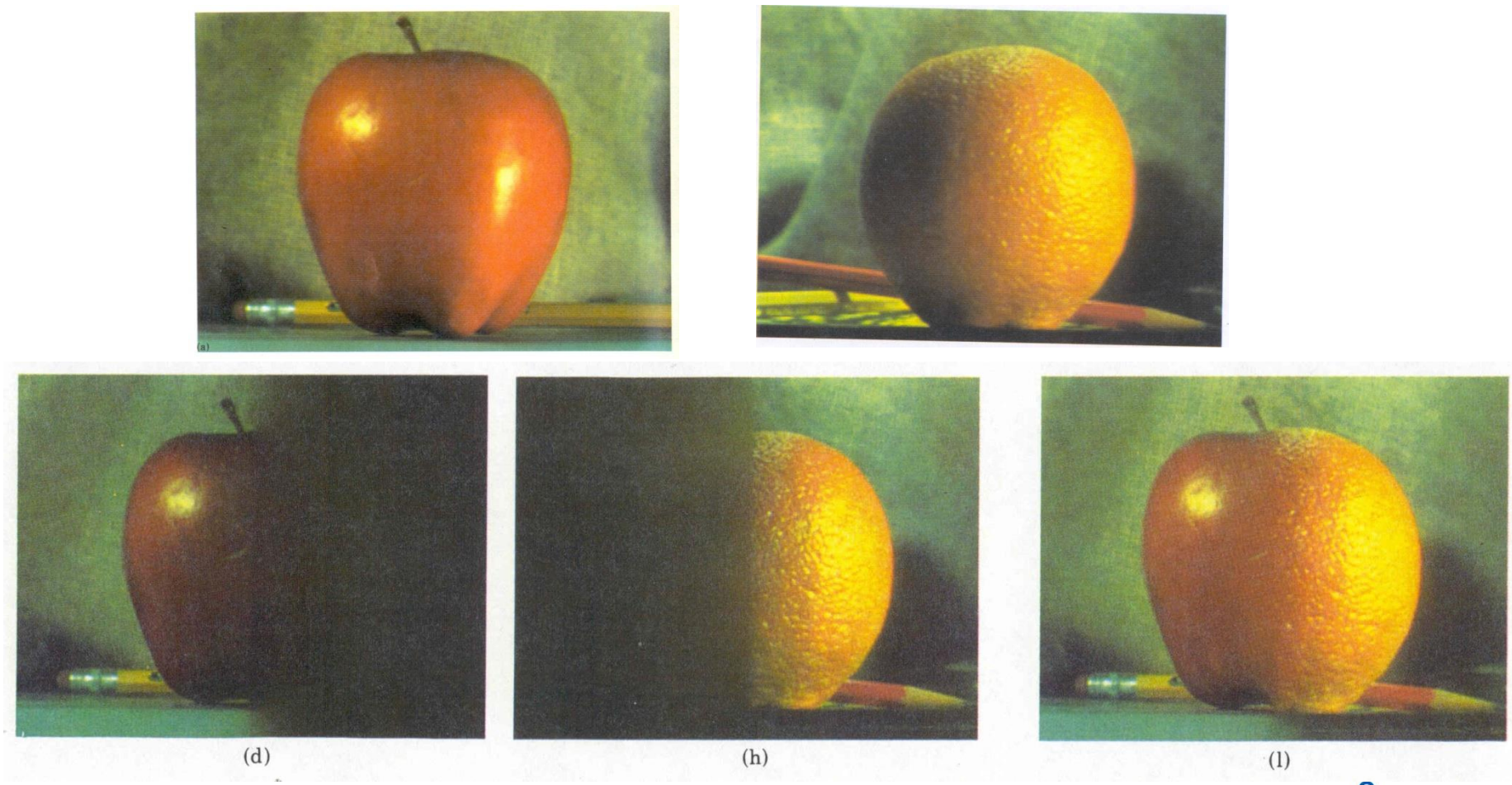
Laplacian level 2

Laplacian level 0



left pyramid | right pyramid | blended pyramid

Pyramid Blending

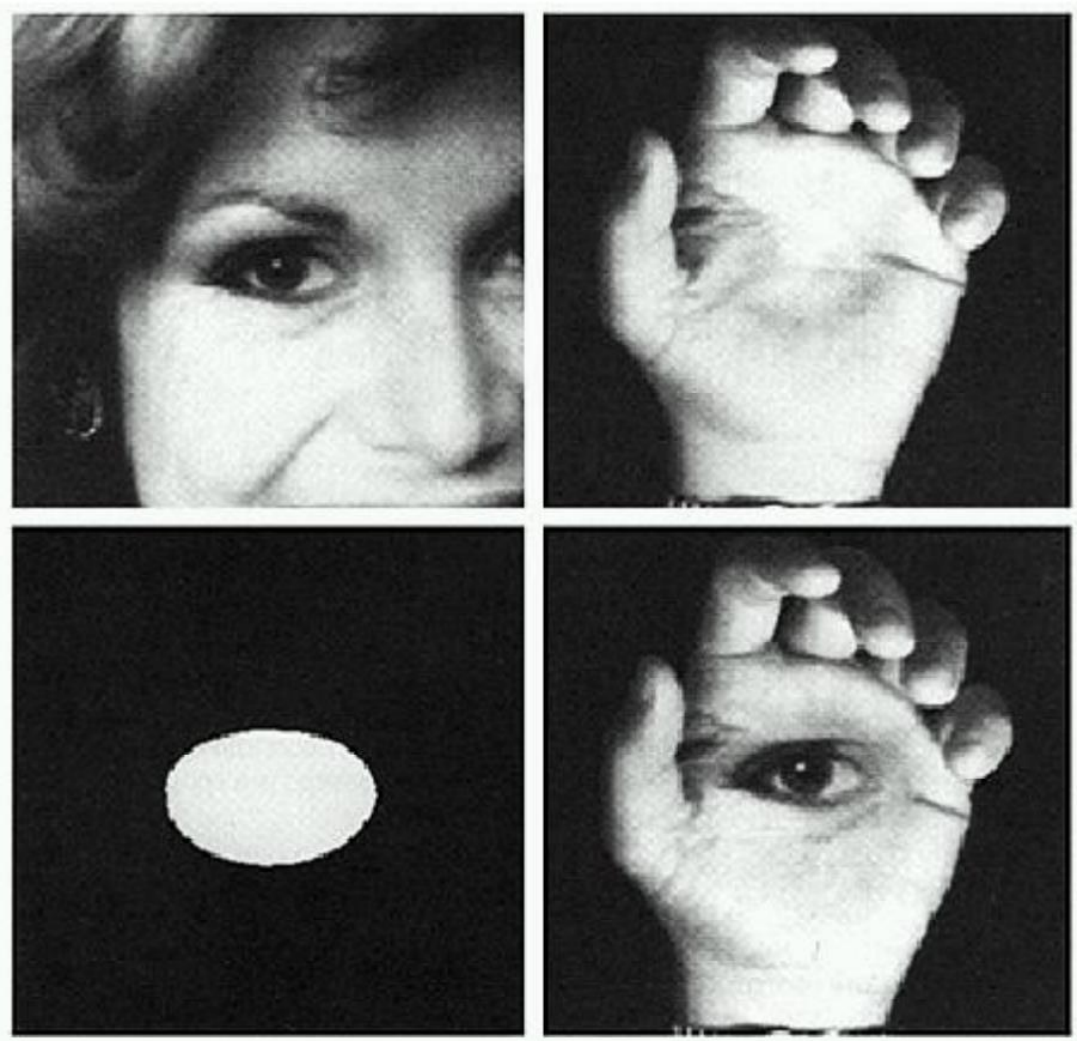


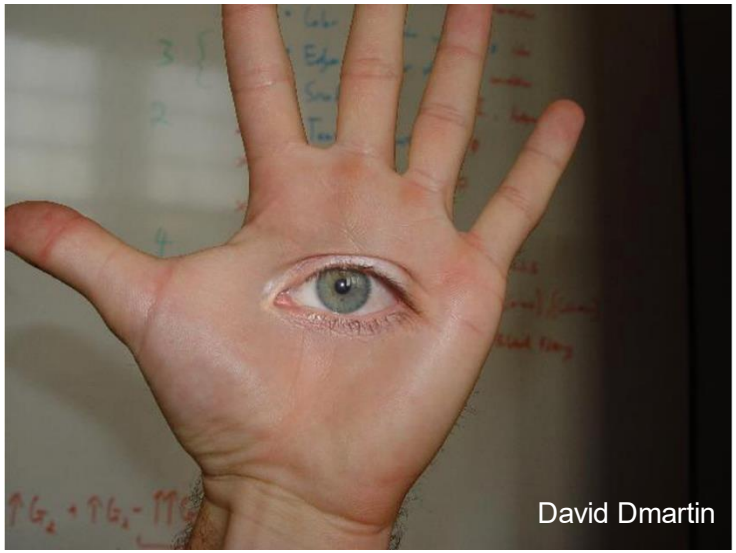
Burt, P. J. and Adelson, E. H., A multiresolution spline with applications to image mosaics, ACM Transactions on Graphics, 42(4), October 1983, 217-236.

Laplacian Pyramid: Blending

- 1. Build Laplacian pyramids LA and LB from images A and B
- 2. Build a Gaussian pyramid GR from selected region R
- 3. Combined pyramid LS from LA / LB using GR as weights:
  - LS(i, j) = GR(i, j) * LA(i, j) + (1 - GR(i, j)) * LB(i, j)
- 4. Collapse the LS pyramid to get the final blended image

Blending Regions





Season Blending

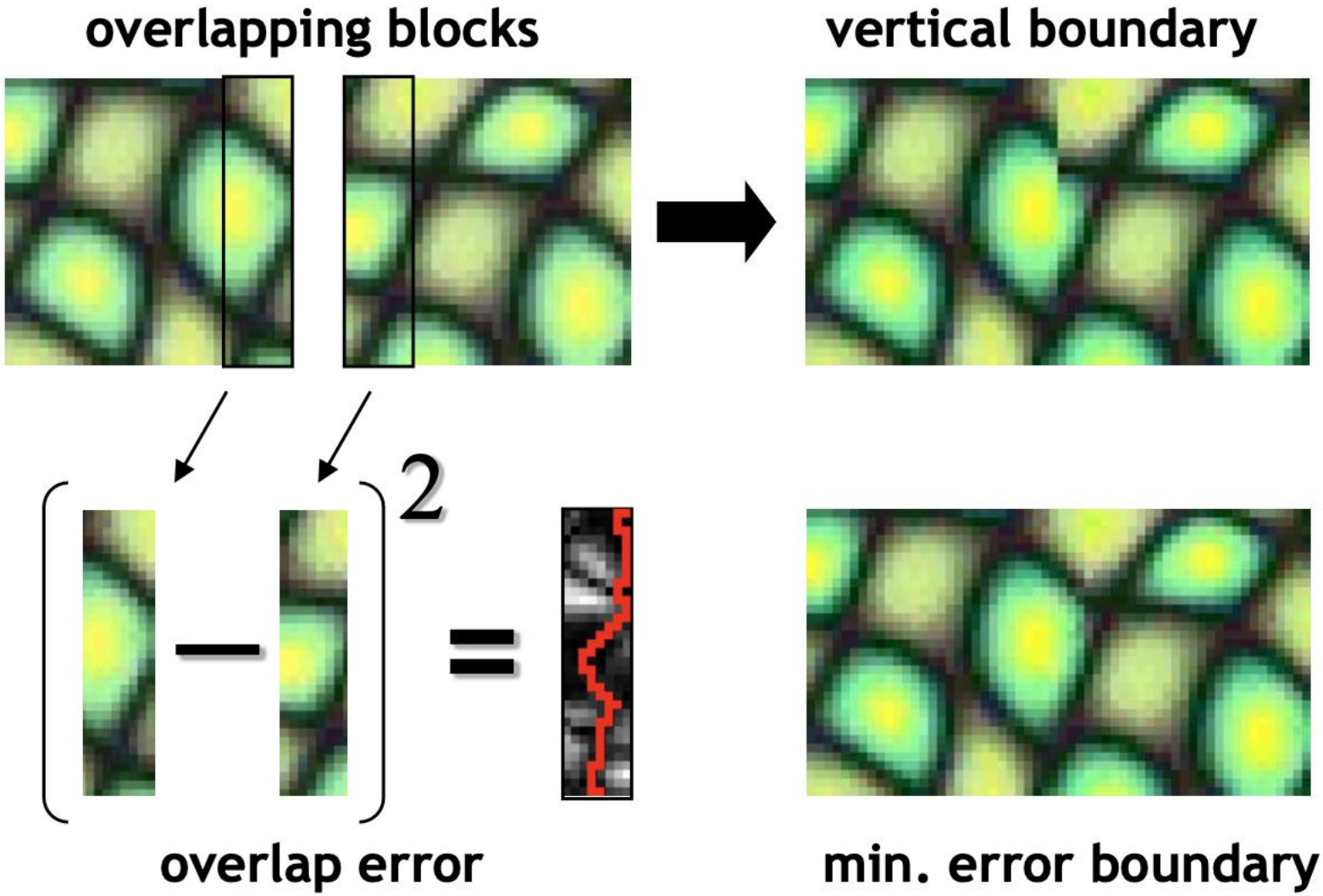
Don’t blend, CUT!

- So far we only tried to blend between two images.
- What about finding an optimal seam?

Don’t blend, CUT!

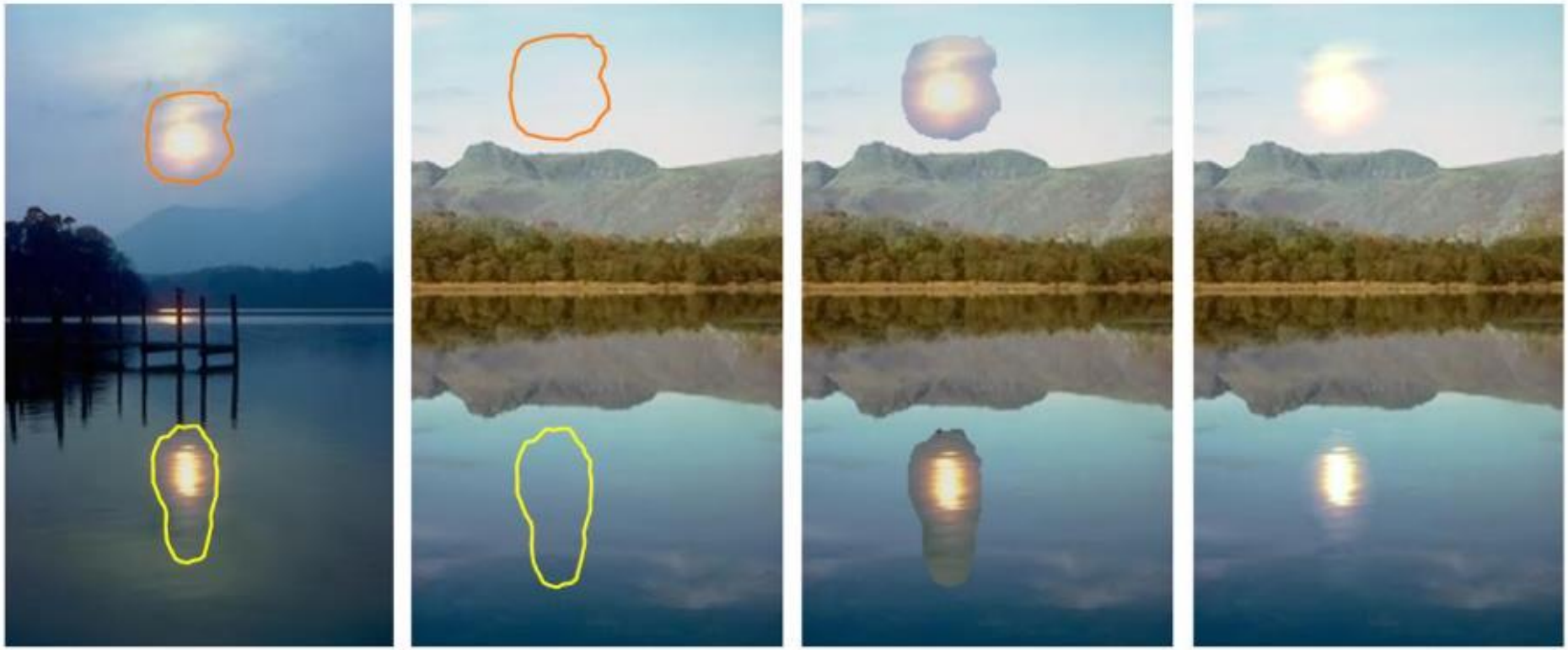
- Segment the mosaic
  - Single source image per segment
- Avoid artifacts along boundaries
  - Dijkstra’s algorithm

Minimal error boundary



Gradient Domain Blending

- Blend the gradients of the two images, then integrate.
  - Focuses on **how colors change** across the image (the gradients) rather than the absolute colors themselves. By matching or modifying these gradients, the final composite looks smoother and more natural.



sources | destinations | cloning | seamless cloning

Perez et al, SIGGRAPH 2003

Gradient Domain Blending

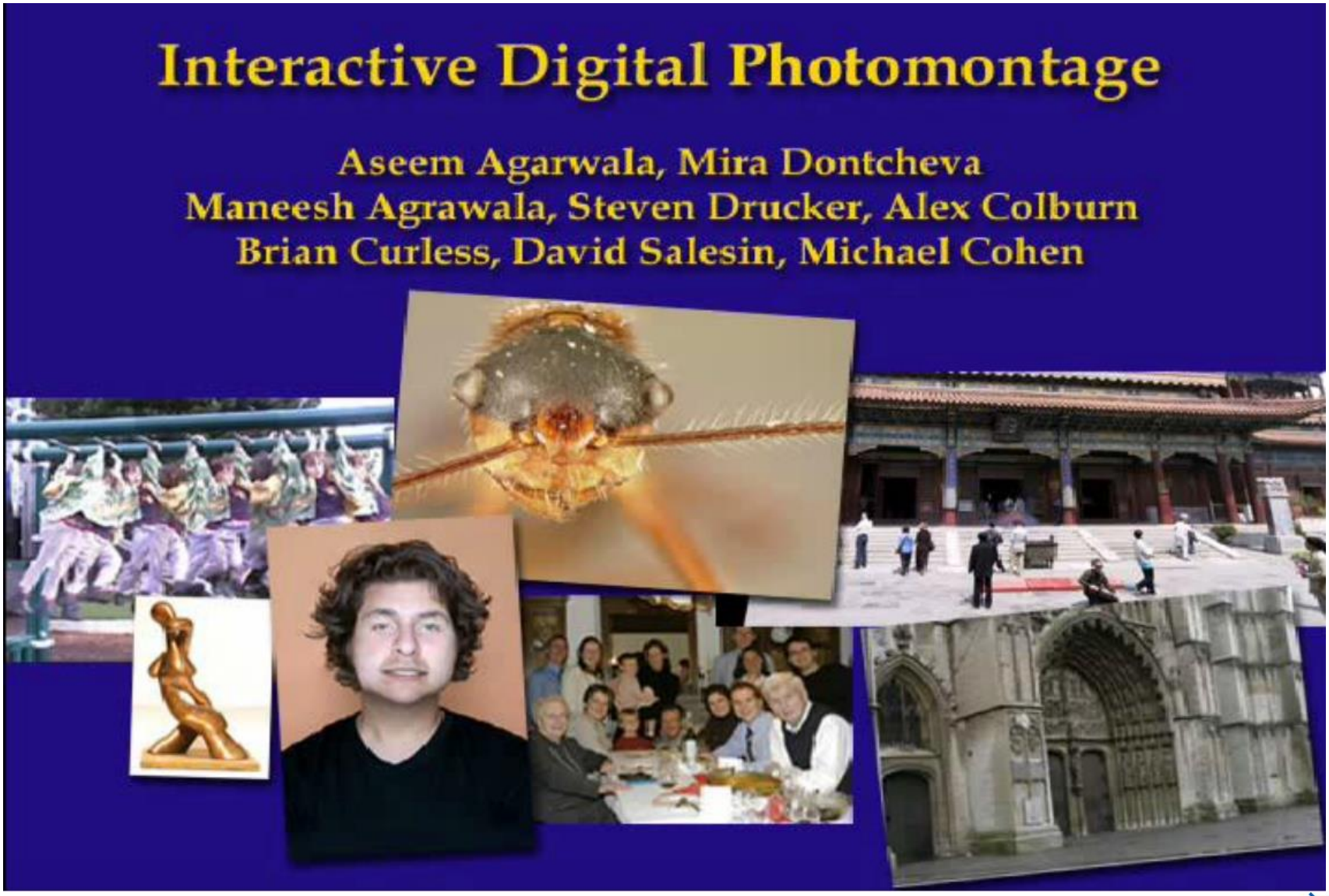
- Blend the gradients of the two images, then integrate.

sources/destinations

cloning

seamless cloning

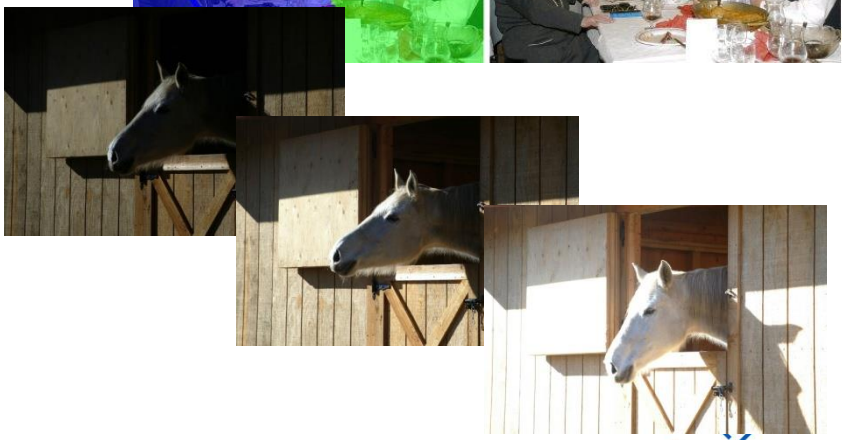
Video (Gradient Domain Fusion)

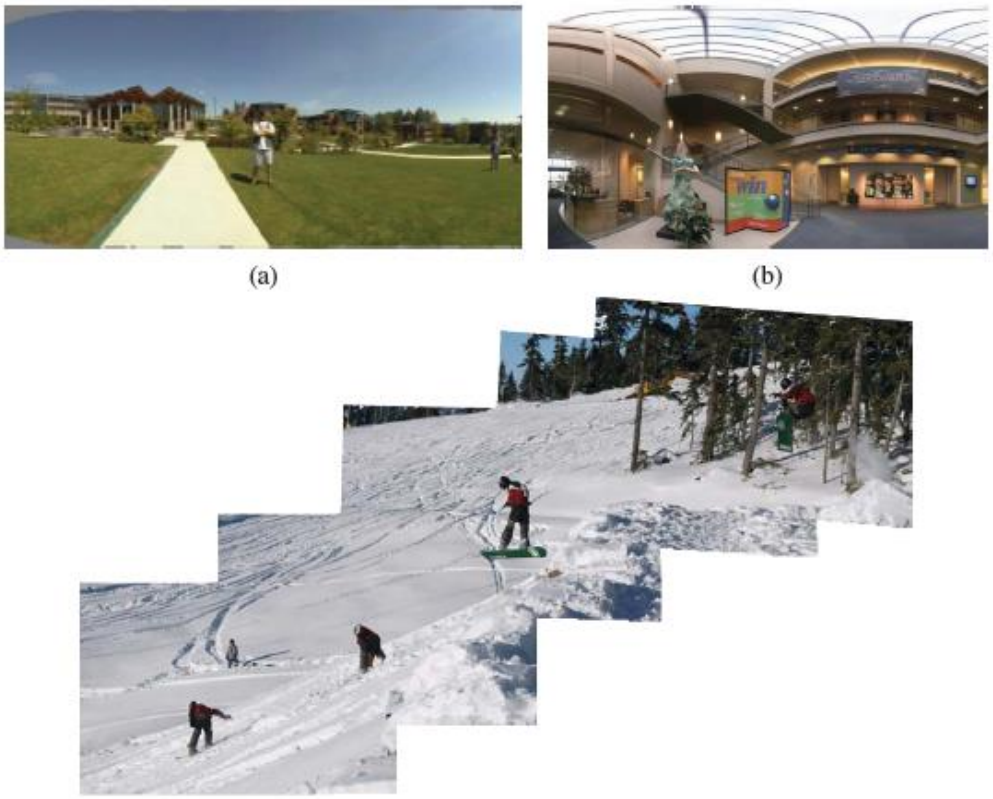


https://youtu.be/Rp7uDRdQRFc

Final thought: What is a “panorama”?

- Tracking a subject
- Repeated (best) shots
- Multiple exposures
- “Infer” what photographer wants?
- Also referred to as “Computational Photography”



Project 2: Stitching

- Feature Detection
- Feature Matching
- Homography
- RANSAC
- Global alignment
- Warping
- Blending